

CHAPTER 1

INTRODUCTION

1.1 Overview

Robotic-assisted surgery has transformed modern surgical practice by improving precision, control, and flexibility. Advanced systems such as the da Vinci Surgical System enable minimally invasive procedures through small incisions, offering high-definition 3D visualization and wristed instruments that replicate the surgeon’s hand movements with exceptional fidelity. Despite these technological advancements, executional errors particularly those related to instrument handling remain a significant challenge, potentially compromising patient safety and surgical outcomes. These errors may include improper needle orientation, repeated attempts at an action, misplacement of instruments outside the endoscopic view, or loss of tool visibility.

Surgical errors are broadly classified into executional and procedural types. Procedural errors occur when gestures are performed in the wrong order or skipped entirely. These errors reflect a deviation from the expected surgical workflow and can be often identifiable using predefined grammar graphs. In contrast, executional errors occur during the performance of surgical gestures and are often subtle and context specific, making them challenging to detect. Consequently, this study focuses on the identification of executional errors from surgical video data.

To address this, recent research has turned toward data driven methods for surgical skill evaluation and error detection. This work utilizes the JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS) a benchmark dataset comprising synchronized video, kinematic data, and gesture annotations recorded during robotic surgical tasks. Focusing on the video modality, this study aims to develop an intelligent system for detecting executional errors in two critical robotic surgical tasks, suturing and needle passing as illustrated in Figure 1.1.



Figure 1.1: Sample frames from the JIGSAWS dataset showing the two robotic surgical tasks used in this study: suturing and needle passing.

The proposed model adopts a hybrid architecture that first extracts spatial features using a pre-trained ResNet-50 and then performs temporal modeling through a combination of 1D Convolutional Neural Networks (1D CNNs), Gated Recurrent Units (GRUs), and self-attention mechanisms. To enhance contextual understanding, the architecture incorporates a dual-pathway structure comprising gesture-aware and gesture-agnostic branches. These pathways are dynamically fused to produce the final prediction, enabling the system to detect executional errors both in the presence and absence of explicit gesture information.

This work contributes to the advancement of automated surgical error detection systems, with the goal of supporting objective skill assessment, real-time feedback, and intraoperative assistance in robotic surgery.

1.2 Motivation

The increasing integration of robotic-assisted surgery into modern healthcare has highlighted both its transformative potential and the unique challenges it presents. While these systems offer remarkable precision and control, they also introduce new complexities particularly in ensuring the surgeon's proficiency and minimizing the risk of executional errors. Even small mistakes in such procedures can lead to serious consequences, such as complications, prolonged recovery, or repeat surgeries. I was also struck by how such procedures are often not covered by insurance, raising questions about perceived risks, accountability, and trust in the technology. These reflections have motivated my interest in exploring how we can better detect and prevent errors in robotic-assisted surgery through intelligent, data-driven systems.

Later, I explored the literature and found that despite technological advances, surgical training and evaluation still rely heavily on subjective observation, which can lead to inconsistent or biased assessments [25]. There is a clear need for intelligent systems that can automatically detect executional errors and understand their context to support more objective skill assessment and safer surgical practice.

1.3 Objective

The primary objective of this project is to develop and evaluate a deep learning framework for detecting executional errors in robotic-assisted surgery using video data from the JIGSAWS dataset.

Key objectives include:

- To model temporal dynamics in surgical tasks through a hybrid sequence learning approach that combines 1D convolutional neural networks (CNNs), gated recurrent units (GRUs), and self-attention mechanisms for better handling of sequential dependencies.
- To design and implement a dual-pathway architecture where one branch focuses on detecting errors in relation to specific surgical gestures (gesture-aware) while the other branch (gesture-agnostic) that captures anomalies irrespective of the gesture being performed.
- To introduce a dynamic fusion mechanism that effectively combines the outputs from both branches, enabling the system to produce robust and accurate predictions that account for both contextual and non-contextual error patterns.

1.4 Scope

This project focuses on the development of a deep learning-based framework for executional error detection in robotic-assisted surgery, using surgical video data from the JIGSAWS dataset as a case study. The scope of the work includes:

- Application of the proposed model to analyze surgical video streams, with an emphasis on detecting and classifying executional errors both in relation to specific surgical gestures and independent of gesture context.
- Exploration of the potential applications of the developed approach in surgical training and intraoperative assistance, particularly in enhancing the accuracy of skill assessment and supporting the delivery of safer surgical care.

This project aims to contribute to the growing field of surgical data science by demonstrating the effectiveness of hybrid deep learning architectures for analyzing complex surgical video data. The findings are expected to have important implications for enhancing surgical training, reducing intraoperative errors, and advancing intelligent feedback systems in minimally invasive procedures.

CHAPTER 2

LITERATURE OVERVIEW

2.1 Overview of Deep Learning Techniques for Robotic Surgical Error Detection

Recent advancements in deep learning have significantly improved error detection in robot-assisted surgery by enabling models to analyze complex spatiotemporal patterns from surgical video and kinematic data. Early approaches primarily relied on handcrafted features and basic recurrent models, which often lacked the capacity to generalize across varied surgical contexts. In contrast, modern deep learning methods leverage transformer architectures, state-space models, and temporal fusion mechanisms to capture both detailed and long-range dependencies within surgical workflows. These models increasingly integrate multimodal data such as endoscopic video, tool trajectories, and gesture annotations to provide more context-aware and accurate error identification. Additionally, gesture-conditioned modeling and predictive capabilities have enhanced the ability to anticipate and localize errors in real time. Despite these advances, a notable research gap remains existing systems often struggle to jointly model generic motion patterns and gesture-specific contexts, which are both critical for reliable error detection. Addressing this gap is essential for developing robust, real-time systems that can assist in surgical training, evaluation, and intraoperative decision-making.

2.1.1 SEDMamba: Enhancing Selective State Space Modelling with Bottleneck Mechanism and Fine-to-Coarse Temporal Fusion for Efficient Error Detection in Robot-Assisted Surgery

The work proposes a framework for video-based surgical error detection in robotic procedures. It begins by extracting spatial features using a pre-trained DINOv2 visual encoder, followed by sequential processing through three Bottleneck Multi-scale State-Space Model (BMSS) blocks. Each BMSS compresses spatial dimensions via 1D convolutions, applies Fine-to-Coarse Temporal Fusion (FCTF) with dilated 1D convolutions (dilation rates 2, 4, 8) to model short- and long-term temporal dependencies, and then expands the features back. A dual-branch structure in each BMSS: a gating branch that bypasses temporal modeling and a selective State-Space Model (SSM) branch that captures long-range temporal patterns. A final convolution generates frame-level error probabilities, trained with binary cross-entropy using OCHRA-annotated labels from the SAR-RARP50 dataset. The model outperforms baselines like MS-TCN, ASFormer, Mamba, and Vim in AUC and average precision, while being more parameter and compute-efficient [10].

2.1.2 A Real-Time Approach for Surgical Activity Recognition and Prediction Based on Transformer Models in Robot-Assisted Surgery

This study introduces a deep learning framework utilizing a modified Transformer architecture to recognize and predict surgical activities in real-time during robot-assisted minimally invasive surgery (RAMIS). The model comprises five fully connected layers, one encoder, and three decoders, specifically designed to handle three primary tasks: gesture recognition, gesture prediction, and end-effector trajectory prediction. Notably, the model operates solely on kinematic data obtained from the joints of the robotic arm, enhancing computational efficiency. Evaluated on the JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS) dataset, the model achieved a gesture recognition accuracy of 94.4%, gesture prediction accuracy of 84.82%, and a minimal trajectory prediction error of 1.34 mm with a 1-second prediction horizon. The computational time per iteration was recorded at only 4.2 ms, demonstrating the model's suitability for real-time applications [11].

2.1.3 Think Step by Step: Chain of Gesture Prompting for Error Detection in Robotic Surgical Videos

The work introduces a novel chain of gesture prompting framework for detecting surgical errors in robotic assisted procedures. Built on the hypothesis that understanding gesture transitions can enhance error identification, the method explicitly models the temporal sequence of gestures to guide the detection process. The study utilizes the JIGSAWS dataset and focuses on suturing and needle passing tasks, where gesture labels and executional error annotations are available. A transformer-based architecture is proposed, where the gesture sequence is used as a prompt to influence the visual representation learning from surgical video frames. The gesture conditioned attention mechanism allows the model to focus on temporally relevant frames and align error prediction with the context of preceding and succeeding gestures. The framework outperforms baseline models that treat gesture and error recognition as independent tasks, showing improved accuracy and robustness in leave-one-user-out cross validation. However, the model's dependency on accurate gesture annotations limits its applicability in real time or unlabeled scenarios, where gesture recognition errors can propagate and affect overall performance. Despite this, the approach provides a more interpretable and sequentially coherent understanding of errors [12].

2.1.4 Keep Your Eye on the Best: Contrastive Regression Transformer for Skill Assessment in Robotic Surgery

The work proposes a novel contrastive regression transformer called “Contra-Sformer” for automated skill assessment in robotic surgery. Rather than predicting an absolute skill score, the model explicitly compares a trainee’s performance video against a reference “best” execution. It uses a ResNet-18 backbone and enhanced temporal convolutions to extract spatiotemporal features from each video, then applies multi-head self-attention to align and contrast the feature sequences of the test and reference videos. In other words, the transformer learns representations of how the input trajectory differs from the exemplar. The training objective is set up to regress the difference in skill score between the test and the expert video, which encourages the model to focus on salient deviations. By integrating attention between two inputs, this contrastive architecture emphasizes relative skill discrepancies [13].

2.1.5 Recognition and Prediction of Surgical Gestures and Trajectories Using Transformer Models in Robot-Assisted Surgery

This research proposes the novel application of transformer models for three tasks in robot-assisted surgery: gesture recognition, gesture prediction, and trajectory prediction. The authors modified the original transformer architecture to generate current gesture sequences, future gesture sequences, and future trajectory estimations using only the current kinematic data of the surgical robot end-effectors. The model was evaluated on the JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS) dataset using Leave-One-User-Out cross-validation, achieving up to 89.3% gesture recognition accuracy, 84.6% gesture prediction accuracy, and a trajectory prediction error of 2.71 mm. This approach enables near-real-time surgical activity recognition and prediction, contributing to improved surgical performance and safety [14].

2.1.6 Runtime Detection of Executional Errors in Robot Assisted Surgery

The work proposes a framework for detecting executional errors during robotic surgery in real time using multimodal surgical data. The study leverages the JIGSAWS dataset, focusing on kinematic and video features collected during standardized training tasks. Executional errors, such as out of view tool movement and multiple failed attempts during needle insertion, are annotated and used to train machine learning models. The proposed method combines handcrafted kinematic features, such as velocities, accelerations, and tool positions, with video-based CNN features to detect deviations in surgical performance. A temporal modeling component, based on recurrent neural networks, captures short term and long-term dependencies in surgeon behavior. The model is evaluated using leave one user out cross validation to assess generalizability across different

surgeon skill levels. Results demonstrate effective runtime detection of errors with high precision and recall, highlighting the model’s potential for real time intraoperative feedback. The study underscores the importance of multimodal data integration and temporal modeling in building practical error detection systems. However, reliance on manually engineered features and limited scalability across complex surgical scenarios remain key limitations. The work contributes to the growing field of intelligent surgical systems that aim to improve safety and training through continuous monitoring [15].

2.1.7 Using 3D Convolutional Neural Networks to Learn Spatiotemporal Features for Automatic Surgical Gesture Recognition in Video

The work proposes using a 3D convolutional neural network to capture spatiotemporal features for surgical gesture recognition in video. It “inflates” a standard 2D ResNet-18 architecture into a 3D CNN that takes stacks of successive frames as input, so that convolutional kernels span both spatial and temporal dimensions. During training, the network learns to label each frame’s surgical gesture by observing a short video clip rather than a single image. In this dense-labeling setup, the 3D CNN outputs a per-frame gesture prediction for each video segment. By leveraging temporal convolutions, the model explicitly encodes motion patterns and tool dynamics in its features, rather than relying on post-hoc temporal models. This spatiotemporal learning strategy allows the network to recognize sequences of surgical actions directly from video, demonstrating that 3D CNNs can effectively capture both appearance and motion for frame-wise gesture classification [16].

2.1.8 Novel Evaluation of Surgical Activity Recognition Models Using Task-Based Efficiency Metrics

The work introduces RP-Net V2, a spatiotemporal neural network for surgical phase recognition that is evaluated with task-based efficiency metrics. The model’s architecture consists of a convolutional front-end that extracts visual features from video snippets, followed by a temporal module that aggregates these features over time. It is trained to label fine-grained surgical substeps (such as suturing phases) from the continuous video stream. Importantly, in addition to standard classification scores, the authors compute task efficiency metrics (e.g., time per step, path length) from the predicted labels. This ties the algorithmic output to practical performance measures. Thus, RP-Net V2 represents a conventional deep learning pipeline (CNN + temporal model) tailored to surgical workflow data, with the novelty of evaluating the impact of activity-recognition accuracy on downstream efficiency metrics [17].

2.1.9 Real Time Context Aware Detection of Unsafe Events in Robot Assisted Surgery

The work presents a real time framework for detecting unsafe events during robot assisted surgery by incorporating context awareness into the model. The authors utilize the JIGSAWS dataset, which includes synchronized kinematic data and gesture annotations from standardized surgical tasks. The approach extracts temporal patterns from the motion data and leverages gesture labels to provide context for identifying executional errors, such as tool collisions and excessive force. The system employs a recurrent neural network to process time series kinematic signals and uses gesture conditioned modules to model the relationship between surgical context and potential safety violations. This allows the model to dynamically adapt to the type of gesture being performed, improving detection accuracy. The framework is evaluated on suturing and needle passing tasks, achieving strong performance in detecting unsafe actions with low latency, which is critical for real time feedback during surgery. The context aware design enhances interpretability by associating specific errors with known gestures. However, the approach relies on accurate gesture labels, which limits its flexibility in real world scenarios where gesture information may be noisy or unavailable. The study highlights the importance of combining temporal dynamics and contextual cues for intraoperative safety, laying the groundwork for more adaptive and interpretable surgical assistance systems [18].

2.1.10 U-NetPlus: A Modified Encoder-Decoder U-Net Architecture for Semantic and Instance Segmentation of Surgical Instrument

The methodology proposed in this work centers around U-NetPlus, a modified version of the original U-Net architecture designed for surgical instrument segmentation in laparoscopic images. The model employs transfer learning by replacing the conventional encoder with a pre-trained VGG-11 or VGG-16 network, which enables the extraction of more robust spatial features from input images. To refine the decoding process, the traditional transposed convolutions used in U-Net are replaced with a nearest-neighbor upsampling technique, followed by standard convolutional layers. This design choice enhances the quality of upsampled feature maps and avoids common artifacts. The model maintains the skip connections characteristic of U-Net, linking encoder and decoder layers at the same resolution to preserve fine-grained spatial information crucial for accurate segmentation. The model is trained and evaluated on laparoscopic video frames, with the architectural choices aimed at improving feature representation, localization precision, and overall segmentation quality [19].

Table 2.1: Literature Overview

SI No.	Year and Publication	Title	Author	Methodology
1	2025, IEEE Robotics and Automation Letters	SEDMamba: Enhancing Selective State Space Modelling with Bottleneck Mechanism and Fine-to-Coarse Temporal Fusion for Efficient Error Detection in Robot-Assisted Surgery	Jialang Xu et al.	The methodology introduces SEDMamba, a dual-branch state-space model combining Bottleneck Multi-scale State-Space blocks and Fine-to-Coarse Temporal Fusion to capture both long-range and local temporal patterns in surgical videos. The model integrates DINOv2-based spatial features for accurate frame-wise error detection, effectively enhancing performance in detecting executional errors during robotic surgery.
2	2025, International Journal of Computer Assisted Radiology and Surgery	A Real-Time Approach for Surgical Activity Recognition and Prediction Based on Transformer Models in Robot-Assisted Surgery	Kai Chen et al.	The study employs a transformer-based architecture with an encoder and multi-decoder design to process kinematic data for gesture recognition, gesture prediction, and trajectory forecasting in real time, supporting surgical workflow analysis and decision assistance during robotic procedures.
3	2024, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)	Think Step by Step: Chain of Gesture Prompting for Error Detection in Robotic Surgical Videos	Hao Lin et al.	The methodology leverages a transformer framework with gesture-conditioned prompting to model sequential dependencies between gestures for enhanced error detection in surgical videos. This design ensures temporal coherence and improves accuracy by integrating gesture context in predicting unsafe or erroneous events.

4	2023, IEEE Robotics and Automation Letters	Keep Your Eye on the Best: Contrastive Regression Transformer for Skill Assessment in Robotic Surgery	Dionysis Anastasiou et al.	The study presents a contrastive regression transformer that combines ResNet and temporal CNN features with contrastive learning to assess surgical skills by comparing expert and trainee video segments. This approach provides interpretable skill scores by focusing on discriminative temporal patterns linked to surgical performance.
5	2022, arXiv preprint arXiv:2212.01683	Recognition and Prediction of Surgical Gestures and Trajectories Using Transformer Models in Robot-Assisted Surgery	Yipeng Jin et al.	The work proposes a transformer-based model to simultaneously perform surgical gesture recognition, gesture prediction, and future trajectory estimation from kinematic data in robotic surgery. This integrated model supports real-time surgical assistance by predicting upcoming surgical actions.
6	2021, IEEE Transactions on Medical Robotics and Bionics	Runtime Detection of Executional Errors in Robot Assisted Surgery	Liang Tao et al.	The approach combines kinematic and video data by extracting handcrafted motion features and CNN-based video features, which are processed using an RNN to model temporal dependencies for real-time executional error detection during robotic surgery.
7	2019, Medical Image Computing and Computer-Assisted Intervention (MICCAI)	Using 3D Convolutional Neural Networks to Learn Spatiotemporal Features for Automatic Surgical Gesture Recognition in Video	Isabell Funke et al.	The methodology utilizes an inflated 3D CNN based on ResNet-18 to learn spatiotemporal features directly from surgical video sequences for automatic gesture recognition, maintaining joint spatial and temporal learning for robust frame-wise classification.

8	2019, Medical Image Computing and Computer-Assisted Intervention (MICCAI)	Novel Evaluation of Surgical Activity Recognition Models Using Task-Based Efficiency Metrics	Syed Zia et al.	The study applies CNN-based spatial feature extraction combined with temporal models for phase recognition in robotic surgery, introducing task-based efficiency metrics such as time-per-step to provide deeper insight into model performance beyond accuracy alone.
9	2019, International Journal of Computer Assisted Radiology and Surgery	Real Time Context Aware Detection of Unsafe Events in Robot Assisted Surgery	Deniz Sarikaya et al.	The work introduces a real-time detection system combining kinematic data and gesture annotations to identify unsafe events during robotic surgery using an RNN framework that incorporates context-aware temporal patterns for recognizing anomalies like collisions and excessive force.
10	2019, Proceedings of the International Conference on Electrical, Computer and Communication Engineering (ECCE)	U-NetPlus: A Modified Encoder-Decoder U-Net Architecture for Semantic and Instance Segmentation of Surgical Instrument	S. M. Kamrul Hasan et al.	The methodology proposes U-NetPlus, a U-Net variant with a VGG encoder and nearest-neighbor upsampling decoder to reduce checkerboard artifacts in surgical instrument segmentation, enhanced by data augmentation for robust performance in diverse scenarios.

Following the literature review, it is evident that existing approaches tend to either rely heavily on gesture annotations or disregard contextual gesture cues altogether, leading to suboptimal performance in dynamic environments. To bridge this gap, the proposed method introduces a unified framework that integrates both gesture-aware and gesture-agnostic pathways. By dynamically modulating between these pathways based on the reliability of gesture information, the approach ensures more robust and adaptive executional error detection, particularly in scenarios where gesture signals may be noisy, ambiguous, or partially missing.

3.2 Data Preparation

The dataset used in this study is the JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS), a benchmark dataset developed by Johns Hopkins University in collaboration with Intuitive Surgical Incorporation [6]. It was collected using the da Vinci Surgical System and contains synchronized video recordings, kinematic measurements, and manual gesture annotations of three robotic surgical training tasks: suturing, needle passing, and knot tying. In this work, only suturing and needle passing are considered, as they include both executional error labels [5] and gesture labels provided by [12], which are critical for supervised learning. The video data, captured using stereo endoscopic cameras, serves as the primary modality for this research, as it preserves the spatial-temporal dynamics essential for accurate error detection and gesture recognition. Each surgical task was performed by eight participants with varying levels of expertise, resulting in a diverse and representative set of recordings. The video clips are provided in AVI format at 30 frames per second, with suturing videos having a resolution of 640×480 and needle passing videos recorded at 320×240 . Each clip ranges in duration from approximately one to three minutes, as shown in Table 3.1.

Table 3.1: Dataset summary of robotic surgical training tasks.

Robotic surgical training tasks	Number of videos	Resolution	Frame per second (fps)	Video format	Duration of video (minutes)
Suturing	39	640*480	30	AVI	~ 1 - 3
Needle Passing	28	320*240	30	AVI	~ 1 - 3

3.3 Data Pre-Processing

To prepare the raw surgical video data for deep learning-based analysis, a structured preprocessing pipeline is applied. Each video is first downsampled to 5 frames per second (fps), striking a balance between preserving temporal dynamics and reducing computational complexity. The downsampled videos are then decomposed into individual frames, each of which is center resized to 224×224 pixels to meet the input size requirements of modern convolutional neural networks. This resizing method focuses on the central region of the frame, where most surgical activity occurs. Following resizing, each frame is normalized using the mean and standard deviation of the ImageNet dataset, ensuring consistency with models such as ResNet-50, which were pretrained on ImageNet. This normalization facilitates stable learning and effective feature extraction. Importantly, each frame is annotated with two labels: a gesture label (from a set of 15 predefined

surgical gestures) and a binary error label indicating the presence or absence of an executional error. These annotations provide the supervision needed for both gesture recognition and error detection tasks.

3.4 Spatial Feature Extraction

Once the video frames are pre-processed, spatial features are extracted using a pre-trained ResNet-50 convolutional neural network, as illustrated in Figure 3.2. This model, trained on ImageNet, is well-suited for learning hierarchical visual representations. The final classification layer is removed, truncating the network at the global average pooling (GAP) layer. Each frame processed through this network yields a 2048-dimensional feature vector that encodes high-level spatial semantics such as tool positioning, tissue interaction, and visual error cues. These vectors effectively capture object boundaries, tool configurations, and other discriminative features necessary to identify subtle execution errors. The extracted features are then flattened and stored sequentially across frames, forming a feature matrix of shape $(T, 2048)$, where T represents the number of frames. By using precomputed features from a pre-trained network, the approach not only accelerates training but also leverages rich representations learned from large-scale image data, enhancing generalization to unseen surgical scenes and alleviating the need to train a spatial encoder from scratch on limited data [8].

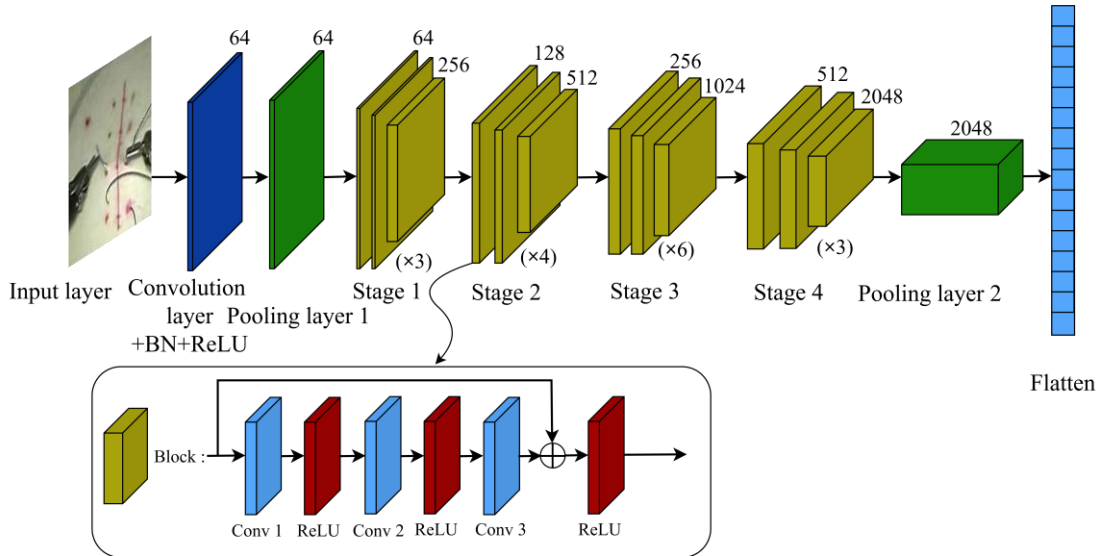


Figure 3.2: ResNet-50 architecture with convolution, pooling, and bottleneck residual blocks across four stages.

3.5 Feature Augmentation

To increase the model’s ability to generalize and suppress overfitting, feature-level augmentation is applied to the spatial embeddings extracted from the ResNet-50 network. This method simulates practical timing noise and spatial variability observed in robotic surgery through controlled variation. Three transformation strategies are employed: Gaussian noise addition, random feature scaling, and temporal jittering. Gaussian noise introduces small, zero-mean perturbations to each frame’s 2048-dimensional feature vector, mimicking sensor-level or visual noise. Random scaling adjusts the magnitude of feature vectors by applying a multiplicative scalar drawn from a uniform distribution, helping the model remain invariant to global activation shifts. Temporal jittering shifts the entire feature sequence forward or backward by a few frames, thereby introducing temporal misalignments that reflect natural variations in gesture timing and execution [23].

These augmentations are applied independently to each training sample in the feature space, preserving computational efficiency while significantly increasing data diversity. As a result, the number of frame-level training samples effectively doubles. Initially, 67 videos across the needle passing and suturing tasks comprised a total of 33,816 frames. After augmentation, the dataset included 67,632 frames. This expansion preserved the original class distribution: needle passing and suturing samples were augmented proportionally, and the ratio of error to non-error frames remained balanced. Specifically, the number of non-error frames increased from 15,508 to 31,016, and error frames from 18,308 to 36,616, as shown in Figure 3.3. By introducing structured variability at the feature level, this augmentation strategy enhances the model’s robustness to noise, intensity shifts, and timing discrepancies ultimately improving the reliability of downstream temporal modeling.

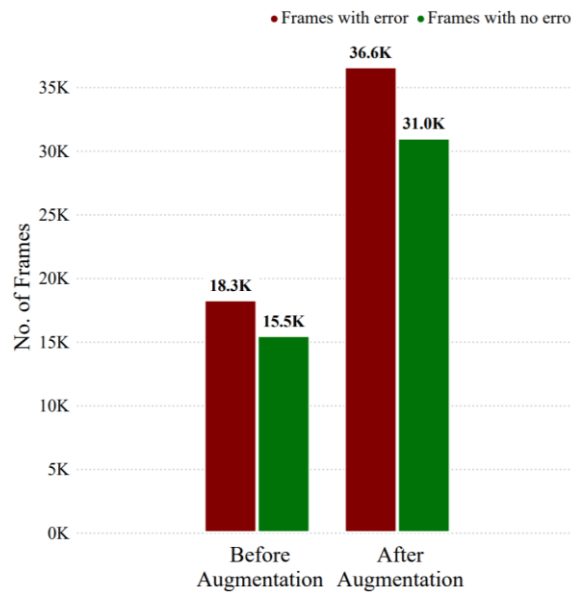


Figure 3.3: Distribution of total frames, frames with error, and frames without error for each task category before and after feature-level augmentation.

3.6 Temporal Modeling

Executional errors in robotic surgery occur over a sequence of actions and are difficult to detect from individual frames alone. To model temporal dependencies, a sequential modeling pipeline is implemented, consisting of three key components: a 1D Convolutional Neural Network (1D CNN), a Gated Recurrent Unit (GRU), and a Self-attention block. The 1D CNN first processes the sequence of spatial features to detect local temporal patterns. Acting as a sliding window over the feature sequence, the 1D CNN captures short term variations, such as abrupt tool motion or repeated gesture attempts, which are early indicators of executional error. The output features are subsequently passed through a Gated Recurrent Unit (GRU), a type of recurrent neural network designed to capture temporal dependencies across sequences. The GRU updates its hidden state at each time step, enabling it to retain important historical information such as the onset of an error or an unsuccessful gesture that may have downstream consequences. However, GRUs inherently tend to emphasize more recent temporal information, which may lead to the attenuation of earlier but critical contextual cues. To address this limitation, a self-attention module is applied on top of the GRU outputs. This mechanism computes pairwise attention scores between all frames in the sequence, allowing the model to assign higher importance to temporally distant but semantically relevant events such as the delayed consequences of an earlier tool misplacement. The combination of GRU-based recurrence and self-attention thus enables the model to construct a rich and context-aware temporal representation of the surgical procedure.

3.6.1 1D Convolutional Neural Network

The 1D Convolutional Neural Network (1D CNN) serves as the first stage of temporal modeling in the detection of executional errors in robotic surgery, playing a crucial role in capturing the local temporal dynamics of surgical procedures. While the input to this module consists of precomputed spatial features extracted from individual video frames, errors in execution are not tied to isolated moments but instead emerge from patterns unfolding over time. The 1D CNN addresses this by applying convolutional filters along the temporal dimension of the feature sequence, effectively acting as a sliding window that moves across consecutive time steps to detect short term variations [22]. Mathematically, the output feature at a given time step t for filter k can be expressed as:

$$y_t^{(k)} = f\left(\sum_{i=0}^{K-1} W_i^{(k)} \cdot x_{t+i} + b^{(k)}\right)$$

where x_{t+i} represents the input feature vector at time step $t + i$, $W_i^{(k)}$ is the weight of the filter k at position i , $b^{(k)}$ is the bias term, K is the kernel size (i.e., the number of consecutive time steps

covered by the filter), and f denotes the ReLU activation function. This equation shows how the 1D CNN computes a weighted sum over a small temporal window of features and passes the result through an activation function to produce a new representation that captures localized temporal patterns. By learning to recognize subtle but critical changes such as abrupt shifts in tool velocity, repeated gestures indicating failed attempts, or sudden deviations in motion trajectories the 1D CNN transforms the sequence into a representation that emphasizes these short-term temporal cues. This localized temporal processing is essential because early indicators of executional errors often emerge over brief intervals, too fleeting to be captured by models that prioritize only long-term dependencies.

Furthermore, the 1D CNN exhibits temporal translation invariance by applying shared filters across the sequence, enabling it to detect local patterns regardless of their position. This is particularly useful given the variability in surgical timing across different procedures. By efficiently filtering and enriching the feature sequence with these localized temporal patterns, the 1D CNN prepares the data for deeper sequential processing by the GRU and self-attention modules, which will integrate this information to capture broader temporal dependencies and long-range contextual relationships. Thus, the 1D CNN lays the foundational layer of temporal understanding, ensuring that critical short-term signals are preserved and highlighted before the model progresses to more complex temporal reasoning.

3.6.2 Gated Recurrent Unit

The Gated Recurrent Unit (GRU) follows the 1D CNN in the temporal modeling pipeline, capturing dependencies over longer time spans in the feature sequence. While the 1D CNN handles short-term variations, the GRU maintains and updates hidden states across time, enabling the model to remember crucial past information such as the onset of an error or a failed gesture that may lead to an execution error. Introduced by Cho et al. in 2014, the GRU is a streamlined RNN variant equipped with two gating mechanisms an update gate and a reset gate to control information flow, making it more efficient and less susceptible to the vanishing gradient problem compared to classic RNN architectures. The update gate regulates how much of the past hidden state is carried forward, while the reset gate determines the blend between the new input and previous state in forming the candidate hidden state [7, 9].

The update gate z_t determines how much of the past information should be carried forward, while the reset gate r_t decides how much of the past state should be discarded. The hidden state h_t at time step t is updated using these gates and the current input x_t (the output from the previous 1D CNN layer), as shown in the following equations:

Update Gate (z_t):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

Here, W_z represents the weight matrix for the update gate, h_{t-1} is the hidden state from the previous time step, x_t is the current input, and b_z is the bias term. The function σ is the sigmoid activation function, which ensures that z_t lies between 0 and 1, controlling how much of the previous state should be kept.

Reset Gate (r_t):

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Similarly, W_r is the weight matrix for the reset gate, and b_r is the bias. The reset gate determines how much of the previous hidden state should be forgotten.

Candidate Hidden State ($\tilde{h}_t$):

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h)$$

The candidate hidden state $\tilde{h}_t$ is computed by combining the reset gate modulated previous state with the current input x_t . The tanh function is used to introduce nonlinearity.

Final Hidden State (h_t):

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

The final hidden state is a combination of the previous hidden state and the candidate hidden state, controlled by the update gate z_t . If z_t is close to 1, the hidden state will largely be influenced by the previous state, if z_t is close to 0, the model will rely more on the new candidate state.

Through these equations, the GRU selectively passes information from earlier time steps and updates its hidden state based on the input and previous context, making it highly effective for learning long term temporal dependencies in sequential data.

3.6.3 Self Attention

The self-attention mechanism forms the final stage of the temporal modeling pipeline, placed on top of the GRU to capture long-range temporal dependencies that the recurrent network alone may miss. While the GRU processes the sequence step by step and is inherently biased toward more recent inputs, self-attention enables the model to directly establish relationships between any two-time steps, regardless of their distance apart. This is especially valuable in robotic surgery, where the impact of an action may not become apparent until several seconds or subsequent gestures later. Self-attention achieves this by computing a weighted sum of all hidden states in the sequence, where the weights reflect each time step's relevance to the current one. This effectively creating shortcut connections across the sequence, allowing the model to better preserve and leverage long-term context [24].

Mathematically, for each time step t , the input hidden state h_t (from the GRU) is transformed into three different vectors: a query q_t , a key k_t , and a value v_t , computed through learned linear projections. These are defined as:

$$q_t = W_Q h_t, \quad k_t = W_K h_t, \quad v_t = W_V h_t$$

where W_Q , W_K , and W_V are learned weight matrices for queries, keys, and values respectively.

For each pair of time steps (t, j) , an attention score is computed by taking the dot product of the query at time t and the key at time j , scaled by the square root of the key dimension d_k to stabilize gradients:

$$\alpha_{t,j} = \frac{q_t \cdot k_j}{\sqrt{d_k}}$$

These scores are then passed through a softmax function to obtain normalized attention weights:

$$\alpha_{t,j} = \text{softmax}(\alpha_{t,j})$$

which ensures that the attention weights across all time steps j sum to 1 for each t .

Finally, the output at time t is computed as a weighted sum of all value vectors:

$$z_t = \sum_{j=1}^T \alpha_{t,j} v_j$$

where T is the total number of time steps in the sequence.

This mechanism enables the model to dynamically focus on the most relevant moments across the entire sequence when producing the representation for each time step. In the context of surgical error detection, self-attention allows the model to link distant events, such as a subtle tool misplacement early in the procedure that later results in an error, or to relate repeated failed gestures across non-consecutive frames. By explicitly modeling these pairwise interactions, self-attention complements the sequential memory of the GRU, overcoming its tendency to prioritize recent information and enabling the model to form a richer, global temporal understanding of the surgical task.

Through this layered architecture starting from local temporal pattern detection with the 1D CNN, progressing to sequential memory with the GRU, and culminating in long range dependency modeling with self-attention the pipeline builds a progressively more comprehensive representation of the surgical procedure, enhancing its ability to detect executional errors that unfold both in the short term and over extended temporal horizons.

3.7 Hybrid Dual Pathway

After the temporal dependencies in the surgical video sequence have been modeled using a combination of a GRU and a self-attention mechanism, the model proceeds to a critical architectural phase designed to refine and specialize its understanding of executional errors. This phase is built upon a hybrid dual-pathway framework, which independently captures both gesture-aware and gesture-agnostic patterns in the data. This split reflects the dual nature of surgical errors: some depend on the specific gesture being performed (e.g., making an incorrect cut during suturing), while others are more general performance indicators (e.g., shaky tool movement) and can be detected regardless of gesture context. To address both types, the model learns simultaneously from two separate views one that incorporates gesture identity (gesture-aware) and another that operates without gesture information (gesture-agnostic). Finally, the outputs of the two pathways are merged via a weighted fusion strategy that emphasizes gesture-relevant cues when reliable, and defaults to general performance signals when gesture information is ambiguous or noisy. This design is inspired by the two-stream architectures used in video action recognition where separate networks model appearance and motion, and fusion yields a complementary representation originally introduced by Simonyan and Zisserman [26].

3.7.1 Gesture Agnostic Path

The gesture-agnostic path of the model operates solely on the temporal features $f_t \in \mathbb{R}^d$ produced by the self-attention block. This pathway is deliberately excluded from gesture label supervision, encouraging the model to learn universal motion-based patterns indicative of executional errors. Such patterns include irregular instrument movement, abrupt pauses, or repetitive motions that may reflect hesitation or correction. These behaviors can signal an error independent of the specific surgical gesture being performed. To detect such patterns, each feature vector f_t is passed through a 1D convolutional layer with kernel size 1, applied independently at each time step. This operation is equivalent to a shared linear classifier that produces a logit score at each frame:

$$\hat{y}_t^{agnostic} = Conv1D_{agnostic}(f_t)$$

This logit represents the model's confidence that an executional error is occurring at time t , based solely on the motion dynamics, without any access to gesture context.

3.7.2 Gesture Aware Path

While gesture-agnostic modeling is valuable, many executional errors are tightly coupled to the context of the gesture being performed. For instance, a hesitation may be acceptable during a transition gesture but critical during a cutting gesture. To capture these context-specific patterns, the gesture-aware pathway in the model incorporates semantic information about the current gesture.

This pathway begins by taking the discrete gesture label $g_t \in \{0, 1, \dots, G-1\}$ at time step t , where G is the total number of unique gesture classes in the dataset. This label is mapped to a dense vector representation via an embedding layer, which is trained jointly with the rest of the model:

$$e_t = Embedding(g_t) \in \mathbb{R}^k$$

The embedding vector e_t encodes latent information about the gesture class, such as its typical motion characteristics and associated risk factors. This gesture embedding is concatenated with the temporal feature vector f_t , producing a joint feature representation:

$$h_t = [f_t; e_t] \in \mathbb{R}^{d+k}$$

This concatenated vector h_t is then passed through a separate 1D convolutional layer designed for the gesture aware path. The convolution again operates over the temporal sequence, enabling the model to detect gesture conditioned anomalies in motion patterns. The resulting prediction from this path is:

$$\hat{y}_t^{aware} = Conv1D_{aware}(h_t)$$

This output captures the likelihood of an executional error at time t , but unlike the gesture-agnostic prediction, it does so by conditioning on the specific gesture being performed. By incorporating gesture semantics into the decision process, the model is better equipped to recognize gesture-specific error patterns such as incorrect tool orientation during suturing or premature grasping during needle passing. Technically, this output is represented as a logit score, a raw value produced by a linear layer that reflects the model's confidence about the presence of an error at that time step, based on both motion features and gesture context.

3.8 Dynamic Fusion

A dynamic fusion mechanism is employed to adaptively combine predictions from two complementary pathways: the gesture-agnostic path, which relies solely on temporal features, and the gesture-aware path, which integrates gesture embeddings with temporal features. While both paths offer valuable insights, their reliability can vary depending on the context at each time step. To determine the relative importance of each path, the model processes the attention-enhanced temporal feature vector $f_t \in \mathbb{R}^d$ through a linear transformation followed by a softmax activation:

$$w_t = softmax(W f_t + b) = [w_t^{agnostic}, w_t^{aware}]$$

where $W \in \mathbb{R}^{2 \times d}$ and $b \in \mathbb{R}^2$ are learnable parameters. This results in a normalized weight vector that governs the relative influence of each prediction path at time step t . The final output is computed as a convex combination of the two logits:

$$\hat{y}_t = w_t^{agnostic} \cdot \hat{y}_t^{agnostic} + w_t^{aware} \cdot \hat{y}_t^{aware}$$

This computation is performed independently at each time step, allowing the model to dynamically adjust its reliance on either path. When gesture cues are weak or ambiguous, the model may favor the agnostic path. Conversely, when gesture information is clear and contextually informative, the aware path receives more weight. This adaptive fusion strategy enhances robustness and improves decision-making, especially in real-time scenarios involving complex and uncertain motion patterns.

3.9 Training Procedure and Optimization

The learning takes place end to end through gradient based optimization, specifically using backpropagation and a loss function defined over the final predictions. The architecture is fully differentiable, which allows the gradients to propagate backward from the final prediction $\hat{y}_t$ through both the gesture agnostic and gesture aware branches, and further back through the self-attention, GRU, and temporal convolutional layers.

At each time step t , the model produces a scalar output $\hat{y}_t \in [0,1]$ indicating the predicted probability of an executional error. This prediction is compared against the ground truth binary label $y_t \in \{0,1\}$, representing whether an error truly occurred at that point in the surgical sequence. Typically, a binary cross entropy loss function is used to quantify the discrepancy between the predicted and true labels. The total loss is computed across all time steps and all training sequences, and this cumulative loss guides the model's parameter updates.

During backpropagation, the gradient of the loss with respect to the final prediction $\hat{y}_t$ is computed and propagated backward. Because $\hat{y}_t$ is a convex combination of the agnostic and aware predictions, the gradients naturally split and flow into both branches according to the current dynamic weights w_t^{agnostic} and w_t^{aware} . This encourages each path to learn how to improve its respective prediction quality. Furthermore, the dynamic weighting mechanism itself, defined by the softmax over $Wf_t + b$, is also learnable. Thus, the model learns not only how to make predictions in each pathway but also when to trust each prediction more, based on the context encoded in f_t .

The gesture aware path also receives gradients with respect to the gesture embeddings, allowing the embedding layer to adjust the vector representation of each gesture class so that it better contributes to error prediction. This means that the gesture embeddings are not static or predefined, they are optimized during training to capture the semantic relationships between gestures and error patterns.

Meanwhile, the gesture agnostic path evolves to detect generic motion cues by relying solely on the features learned from the GRU and self-attention blocks. Because it does not use gesture labels, it acts as a form of regularization and provides a fallback when gesture information is ambiguous.

3.10 Model Implementation and Evaluation

The proposed model was implemented using the PyTorch deep learning framework and trained on a laptop configured with an NVIDIA GTX 1650 GPU and 8 GB RAM. The model optimization was performed using the Adam optimizer with a learning rate of $7e^{-5}$, trained over 25 epochs with a batch size of 1.

To ensure subject-independent evaluation, a 5-fold Group K-Fold cross-validation strategy was adopted. This approach utilizes unique subject identifiers to generate non-overlapping folds, ensuring that frames from the same subject do not appear in both the training and validation sets. This setup enhances the generalizability of the model to unseen subjects.

The model was trained using the binary cross-entropy loss function, and a sigmoid activation was applied to the final layer to generate frame-wise probability scores indicating the likelihood of an executional error.

3.10.1 Evaluation Metrics

To comprehensively assess the effectiveness and reliability of the proposed classification model, several evaluation metrics were employed. These metrics provide insights into the model's performance across various aspects of classification quality. The evaluation metrics used include the confusion matrix, accuracy, precision, recall, F1 score, ROC curve (AUC), and Jaccard Index. Below is a detailed explanation of each metric along with its corresponding formula.

Confusion Matrix

The confusion matrix is a table that describes the performance of a classification model by displaying the actual versus predicted classifications. It consists of four components:

- True Positives (TP): The number of correctly predicted positive samples.
- True Negatives (TN): The number of correctly predicted negative samples.
- False Positives (FP): The number of incorrectly predicted positive samples.
- False Negatives (FN): The number of incorrectly predicted negative samples.

Accuracy

Accuracy measures the proportion of correctly classified frames (both error and non-error) among the total number of frames. It provides an overall sense of how well the model is performing.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision

Precision is the ratio of true positive error predictions to all frames predicted as errors. It shows how accurate the model is when it flags a frame as an error.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Recall measures the ratio of correctly predicted error frames to all actual error frames. It reflects the model's ability to catch true errors.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score

The F1 Score is the harmonic mean of precision and recall. It provides a balanced evaluation, especially useful when the dataset has an uneven distribution of error and non-error frames.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Jaccard Index

The Jaccard Index measures the similarity between the predicted and actual sets of error frames. It is defined as the size of the intersection divided by the size of the union of the predicted and actual positive instances:

$$\text{Jaccard Index} = \frac{TP}{TP + FP + FN}$$

By employing this combination of evaluation metrics, the performance of the model can be rigorously assessed across various aspects, including correctness, sensitivity, precision, and robustness to imbalanced labels. This evaluation setup ensures that the model is not only accurate but also reliable in practical, subject-independent surgical error detection scenarios.

CHAPTER 4

RESULTS AND DISCUSSIONS

This section presents and discusses the results obtained from evaluating the proposed model. The evaluation focuses on the model's ability to detect executional errors at the frame level using metrics such as the confusion matrix, accuracy, precision, recall, F1 score, AUC, and Jaccard Index. The results are visualized through detailed plots and statistical summaries to provide a comprehensive understanding of the model's performance

4.1 Training performance across the folds

To evaluate the robustness and generalization capability of the proposed model, a 5-fold cross-validation strategy was employed. In this approach, the dataset was divided into five equal parts, where in each iteration, four subsets were used for training while the remaining one was reserved for validation. This ensured that every data sample contributed to both training and validation phases, offering a more reliable estimate of model performance.

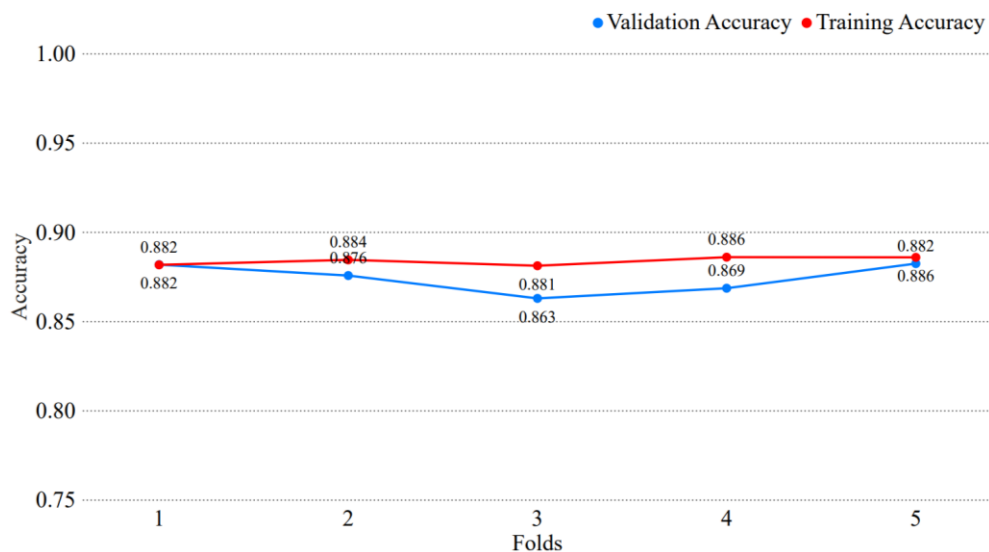


Figure 4.1: Training and validation accuracy across the five folds of cross-validation.

As shown in Figure 4.1, the training accuracy remained consistently high, ranging around 88–89%, while the validation accuracy closely followed, ranging between 86% and 88%. This consistency across folds indicates that the model generalizes well to unseen data and is not significantly overfitting. A slight dip in validation accuracy for Fold 3 may suggest variability in data distribution, but overall, the results remain stable and robust.

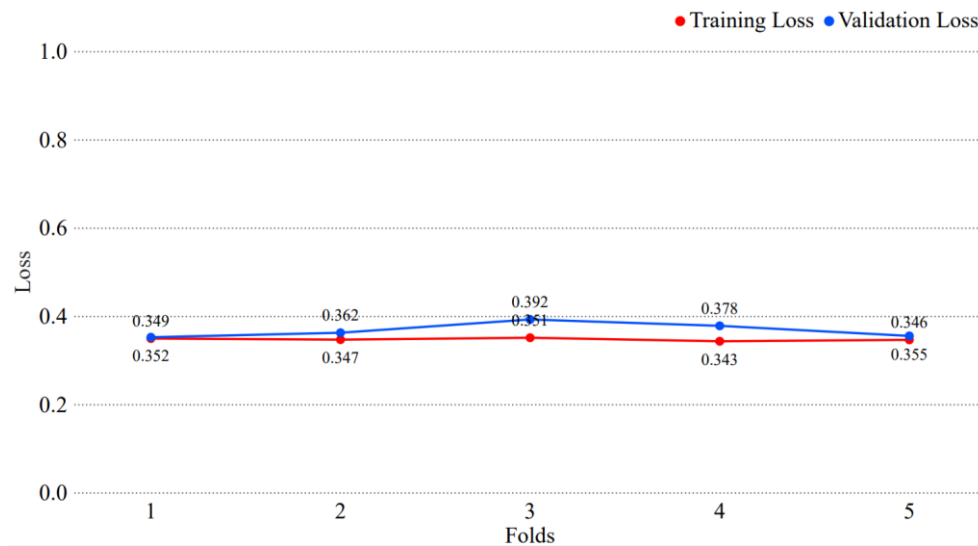


Figure 4.2: Training and validation loss across the five folds of cross-validation.

The corresponding training and validation loss values are illustrated in Figure 4.2. Both loss curves exhibit similar patterns, with values remaining in the range of approximately 0.33 to 0.39. The small and consistent gap between training and validation loss further confirms that the model maintains a balanced learning process, without excessive reliance on the training data. The minor peak in validation loss observed in Fold 3 aligns with the dip in validation accuracy, reinforcing the observation of fold-specific variation.

4.1.1 Training performance on a sample fold

To gain deeper insight into the learning dynamics of the model, the training and validation accuracy and loss were tracked across 25 epochs on a representative fold from the 5-fold Group K-Fold cross-validation. The performance trends observed during this fold are presented in Figure 4.3, highlighting how the model optimizes over time and balances generalization with fitting.

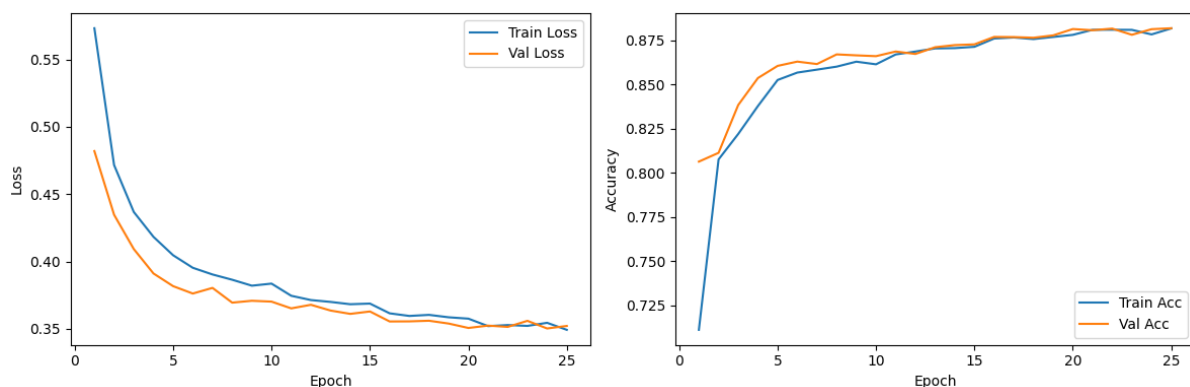


Figure 4.3: Training and validation loss and accuracy curves for a representative fold

The loss curves (left panel) show a clear and smooth downward trajectory for both training and validation sets. The training loss steadily decreases from approximately 0.57 to around 0.35, while the validation loss follows a similar path, settling just below 0.35 by epoch 25. The close alignment

between the training and validation loss curves, particularly after epoch 10, suggests strong generalization and minimal overfitting.

The accuracy curves (right panel) reveal rapid improvement during the initial training phase. Training accuracy rises sharply from approximately 71% in the first epoch to over 85% by epoch 25. Validation accuracy also increases quickly, surpassing 85% early on and gradually approaching 88% by the final epoch. Notably, the validation accuracy consistently tracks above the training accuracy throughout the training process, reinforcing the model's robustness and stability.

Together, these results indicate that the model demonstrates effective and stable convergence behavior. The consistent reduction in loss and the synchronized improvement in accuracy underscore the model's capacity to learn meaningful patterns for executional error detection in surgical tasks without signs of overfitting.

4.2 Model prediction quality on a representative validation fold

To assess the frame-level prediction quality of the model, the model's output was visualized against the corresponding ground truth annotations for a representative test sequence from a validation fold. The results are shown in Figures 4.4 and 4.5.

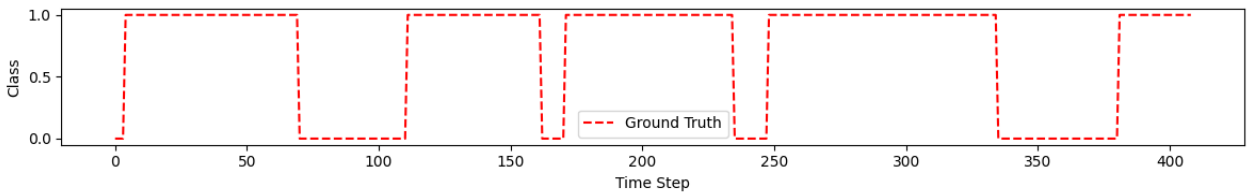


Figure 4.4: Ground truth binary annotations of executional errors across a sample test sequence.

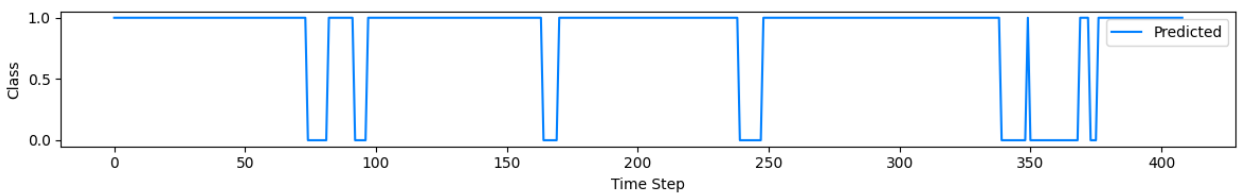


Figure 4.5: Model-predicted executional error sequence for the same test sample.

Figure 4.4 presents the ground truth labels, where a class value of 1 (highlighted by red dashed lines) indicates the presence of an executional error, and 0 denotes normal surgical activity. The temporal segmentation reveals discrete error events with sharp transitions, capturing the dynamic nature of human surgical behavior.

Figure 4.5 illustrates the model's predicted output in blue. The predictions closely track the ground truth signal, accurately identifying both the onset and offset of error phases. The alignment

between predicted and actual error regions is notably precise, with the model effectively suppressing false positives in non-error segments and showing high sensitivity to error transitions.

4.3 Dynamic weighting across time steps for the representative fold

The model incorporates both gesture-aware and gesture-agnostic paths, fused dynamically based on context. The graph below illustrates the time-varying weights assigned to each path during inference on the same test sequence.

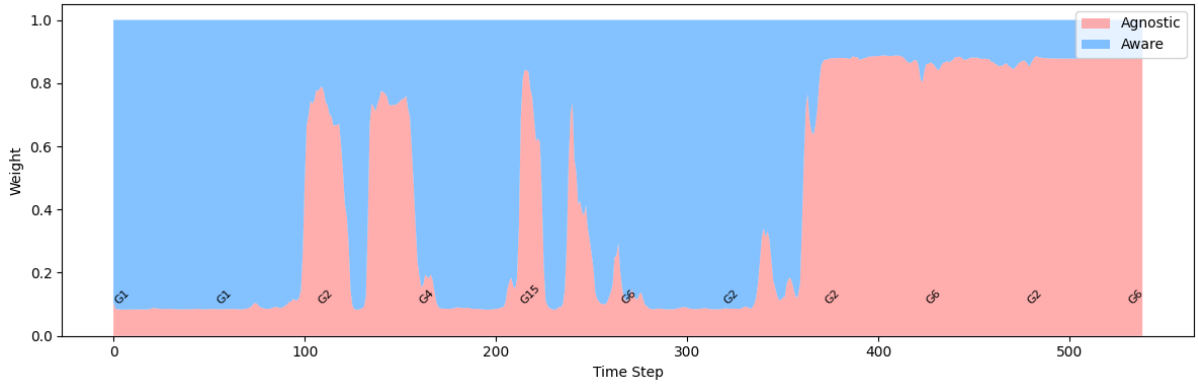


Figure 4.6: Dynamic weighting across sequence for the test sample.

The model adjusts weight allocation based on the active gesture class. For well-defined gesture segments, the gesture-aware path is more trusted (highlighted in blue). During ambiguous transitions or noisy segments, more weight shifts to the gesture-agnostic path (highlighted in pink), enabling robust detection without relying on gesture annotations. This adaptability is crucial for ensuring reliable predictions, even when gesture labels are missing or uncertain.

4.4 Confusion matrix analysis on a representative fold

To further evaluate the classification performance of the model, a confusion matrix was generated for a representative fold from the 5-fold cross-validation, as shown in Figure 4.7. The matrix summarizes the number of correct and incorrect predictions for each class.

From the figure, it can be observed that the model correctly classified 10,437 instances of class 0 and 10,732 instances of class 1. However, 1,290 class 0 instances were misclassified as class 1, and 1,549 class 1 instances were misclassified as class 0. These values indicate that the model performs well in distinguishing between the two classes, achieving a high level of precision and recall for both.

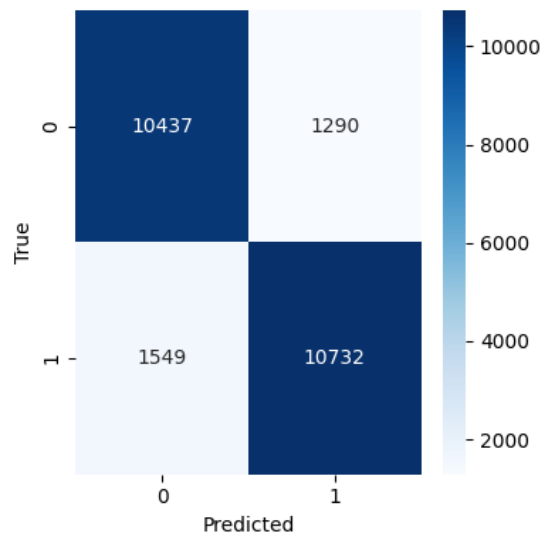


Figure 4.7: Confusion matrix of a sample fold.

4.5 Evaluation metrics across folds

To provide a comprehensive assessment of the model classification performance, multiple evaluation metrics were computed across all five folds of the Group K-Fold cross-validation. These metrics include Jaccard index, precision, recall, F1 score, and area under the ROC curve (AUC), and are illustrated in Figure 4.8.

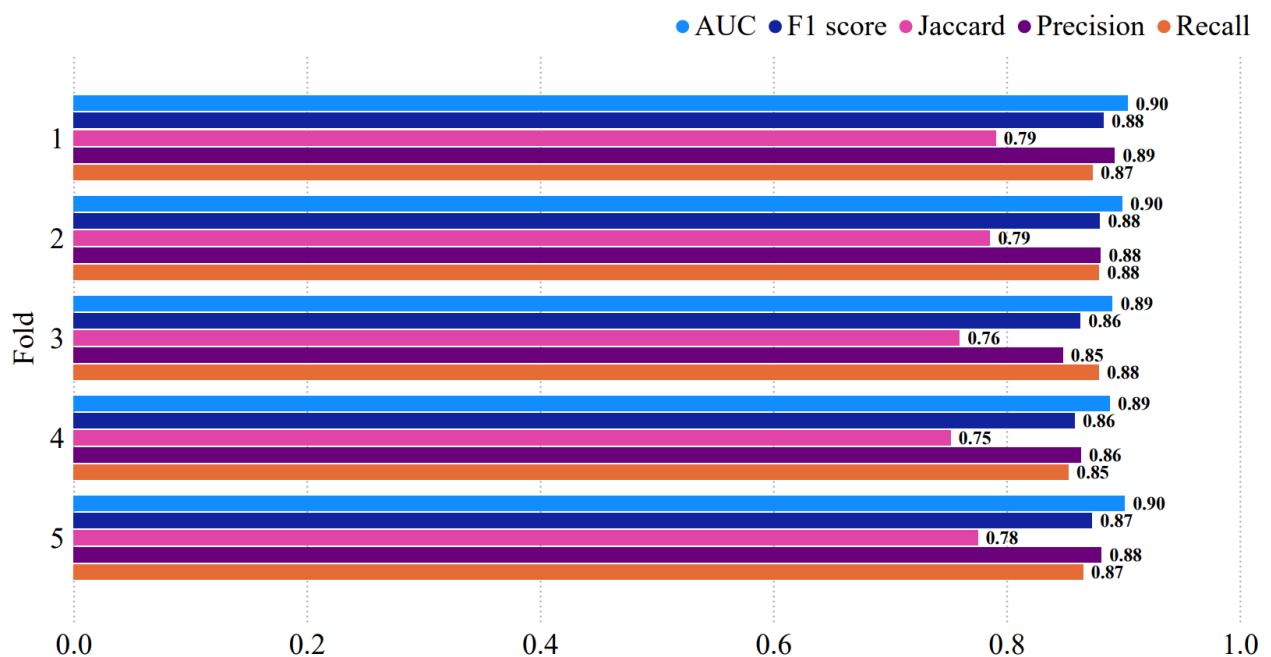


Figure 4.8: Evaluation metrics across the five folds of Group K-Fold cross-validation.

The results indicate consistently strong performance across all folds. Precision and recall values remain high and balanced, reflecting the model's ability to correctly identify both positive and negative cases without significant bias. The F1 score, which considers both precision and recall, further confirms this balance. AUC values are uniformly high across folds, suggesting excellent

separability between the two classes. The Jaccard index, although slightly lower than the other metrics due to its stricter penalization of false positives and false negatives, still remains at an acceptable level, reinforcing the model's robustness.

Overall, the model maintains stable and high classification performance across all evaluation metrics and folds, demonstrating its reliability and generalization capability for surgical error detection tasks.

4.5.1 Averaged Performance Metrics

To summarize the overall effectiveness of the model, key performance metrics were averaged over the five folds of Group K-Fold cross-validation. The results, presented in Table 4.1, provide a quantitative overview of the model's classification performance and its stability across folds.

Table 4.1: Model performance metrics averaged over 5-Fold group K-Fold cross validation

Metric	Value $\pm$ Std. Dev.
F1 Score	0.8727 ± 0.0098
Accuracy	0.8742 ± 0.0076
Jaccard Index	0.7727 ± 0.0149
AUC	0.8968 ± 0.0060
Precision	0.8734 ± 0.0156
Recall	0.8702 ± 0.0099

The model achieved an average F1 score of 0.8727 ± 0.0098 , indicating a strong balance between precision and recall. The overall accuracy was 0.8742 ± 0.0076 , reflecting the model's ability to correctly classify the majority of instances. AUC reached 0.8968 ± 0.0060 , confirming the model's excellent discrimination ability between classes. Precision (0.8734 ± 0.0156) and recall (0.8702 ± 0.0099) remained closely aligned, suggesting the model is not biased toward false positives or false negatives. The Jaccard index, though typically more conservative, recorded a strong value of 0.7727 ± 0.0149 , reinforcing the model's reliable performance.

4.6 Comparison with Existing State of the Art Methods

To assess the effectiveness of the model in surgical error detection, the recently proposed Chain-of-Gesture (COG) model [12] was re-implemented following the details provided in the original publication. COG represents a state-of-the-art approach that leverages gesture-aware prompting and multi-scale temporal reasoning within an end-to-end transformer-based architecture. For a fair evaluation, both models were trained and validated on the JIGSAWS dataset using consistent preprocessing and evaluation protocols. In contrast to COG, the proposed model adopts a streamlined architecture optimized for frame-level classification, relying on fine-grained spatiotemporal features without the multi-path reasoning modules.

Both models were evaluated on the JIGSAWS dataset, though under different validation schemes: COG was tested using Leave One Supertrial Out (LOSO) cross-validation, while the proposed model was evaluated with 5-fold Group K-Fold cross-validation. Despite the difference in setup, a meaningful frame-level performance comparison is still valid due to the shared dataset and task formulation.

Table 4.2: Frame-level performance comparison between the proposed model and the COG model on the JIGSAWS dataset.

Method	F1 Score	Accuracy	Jaccard
COG*[12]	0.7719	0.7188	0.6285
Proposed model	0.8727	0.8742	0.7867

As shown in Table 4.2, the proposed model demonstrates superior frame-level error detection performance, marked by notable improvements in F1 score, accuracy, and Jaccard. These results reflect the model’s ability to reduce false positives and maintain strong generalization across validation folds.

In addition to its improved classification performance, the proposed model also exhibits strong computational efficiency. The average inference time per frame was measured at 7.71 milliseconds, with a minimum of 5.93 milliseconds, corresponding to an average throughput of approximately 129.78 frames per second (FPS) across multiple surgical video sequences. This high-speed inference highlights the model’s suitability for real-time deployment, particularly in surgical settings where both accuracy and low-latency decision-making are essential.

4.7 Cross-Domain Evaluation on SAR-RARP50 Dataset

To evaluate the model on real surgical videos, the SAR-RARP50 dataset was used. This dataset comprises 50 robot-assisted radical prostatectomy (RARP) surgeries, recorded in real operating rooms, and includes detailed frame-level annotations of surgical phases, activities, and executional errors. SAR-RARP50 is currently one of the most realistic and comprehensive datasets available for robotic surgery research. For evaluation, the binary error labels provided by the SEDmamba study [10], which identified executional errors across time using expert feedback, were used as ground truth to assess the model’s ability to detect errors during surgery.

Table 4.3: Surgical error detection performance on SAR-RARP50

Metric	Value
Accuracy	0.5139
Precision	0.5097
Recall	1.0000
F1 Score	0.6752
AUC	0.5931

The evaluation results, summarized in Table 4.3, show that the model achieved an accuracy of 51.39%, a precision of 50.97%, a recall of 100.00%, an F1 score of 67.52%, and an AUC of 0.5931. These results indicate that the model is highly sensitive to detecting executional errors, successfully identifying nearly all error frames. However, this comes at the cost of over-predicting errors, as reflected in the relatively modest precision score.

While the model misses very few error frames, its tendency to classify normal frames as errors leads to a higher false positive rate. This could hinder its practical use in surgical environments, where false alarms may cause unnecessary concern or disruption. One key factor is the significant domain shift between datasets. The model was trained on the JIGSAWS dataset, which features lower resolution (e.g., 320×240), visually consistent surgical scenes. In contrast, SAR-RARP50 contains high-resolution (e.g., 1920×1080), complex, and varied footage. Although frames were resized to 224×224 for input, this can lead to loss of fine-grained spatial cues such as tool tips or subtle tissue interactions, which are crucial for accurate error detection. These rich contextual features, more prevalent in SAR-RARP50, may not be effectively preserved, especially given the model’s prior exposure to the more uniform and lower-resolution domain of JIGSAWS.

Refinements to the architecture for high-resolution input can help the model better handle complex visual details. Additionally, domain adaptation strategies may reduce the shift between the training environment and real-world surgical settings.

CHAPTER 5

CONCLUSION AND FUTURE DIRECTIONS

5.1 Conclusion

This project explored the integration of gesture-aware and gesture-agnostic deep learning techniques to enhance the detection of executional errors in robotic-assisted surgery, with a focus on video data from the JIGSAWS dataset. The primary objective was to develop a model that leverages both general spatiotemporal patterns and gesture-specific contexts to improve error detection accuracy. To achieve this, a hybrid architecture was designed based on a pre-trained ResNet-50 for spatial feature extraction, coupled with a temporal modeling pipeline incorporating 1D Convolutional Neural Networks (1D CNNs), Gated Recurrent Units (GRUs), and Self-Attention mechanisms. These components enable the model to effectively capture local and long-range temporal dynamics, resulting in precise identification and classification of errors in suturing and needle-passing tasks.

The study involved a detailed analysis of surgical video data, evaluated through rigorous 5-fold Group K-Fold cross-validation on JIGSAWS. The model demonstrated robust performance across key evaluation metrics including accuracy, precision, recall, F1 score, AUC, and Jaccard Index confirming its effectiveness in detecting executional errors within the training domain. Additionally, the dynamic weighting mechanism proved instrumental in balancing contributions from the gesture-aware and gesture-agnostic pathways, enhancing adaptability in scenarios with noisy or absent gesture annotations. These findings underscore the significant potential of combining hybrid deep learning architectures with surgical video analysis to advance the precision and reliability of error detection.

However, when the model was evaluated on the high-resolution SAR-RARP50 dataset, performance dropped significantly despite fine-tuning. This decline is primarily attributable to the domain shift between the two datasets: JIGSAWS comprises low-resolution, visually consistent surgical scenes, whereas SAR-RARP50 features high-resolution, visually complex, and diverse real-world procedures. While input frames were resized to 224×224 to match the model's expected input size, this uniform scaling may still have degraded fine-grained spatial details such as tool tips, interaction zones, or subtle anatomical landmarks that are more prominent and informative in high-resolution data. These findings underscore the importance of architectural adaptations or domain adaptation techniques to ensure robust generalization across diverse surgical environments.

5.2 Suggestions for Future Work

The promising outcomes of this project highlight the potential of gesture-aware and agnostic deep learning for surgical error detection. However, several avenues for further research and development could enhance the model's applicability, accuracy, and impact in clinical settings. The following suggestions outline potential directions for future work:

- **Expansion of Dataset:**

Expanding the dataset to encompass a wider variety of surgical tasks and error types could improve the model's generalizability. Including diverse procedures beyond suturing and needle-passing would enable the development of more robust models capable of addressing the complexities and variability of different surgical contexts.

- **Development of Real-Time Monitoring Systems:**

Advancing the approach into a real-time monitoring system could transform surgical error detection by enabling dynamic performance assessment during procedures. Such systems would facilitate immediate feedback, potentially reducing complications and improving surgical outcomes in live settings.

- **Field Trials and Real-World Applications:**

Conducting field trials in actual surgical environments is critical to evaluate the practical utility of the model. Testing across diverse settings would reveal how factors like lighting, camera angles, and surgical variability affect performance, paving the way for effective clinical implementation.

- **Domain Adaptation & Multi-Resolution Modeling:**

Incorporate multi-resolution feature modeling or domain adaptation techniques to mitigate domain shift between low-resolution datasets and high-resolution, real-world datasets, ensuring better generalization and preservation of critical spatial context.

Future directions present valuable opportunities to enhance gesture-based deep learning models by improving their adaptability to diverse surgical environments. Advancements such as multi-resolution modeling, domain adaptation, and context-aware preprocessing could significantly improve model robustness, leading to greater surgical precision, enhanced safety, and more effective training outcomes in real-world clinical settings.

REFERENCES

- [1] Zhen Shen, Yifan Zhang, and Xiaofei Wang. “Human Activity Recognition Using Gated Recurrent Attention Neural Network,” *IEEE Access*, volume 7, pages 108398–108409. 2019.
- [2] Twinanda A. P. et al., “EndoNet: A Deep Architecture for Recognition Tasks on Laparoscopic Videos,” *IEEE Trans. Med. Imaging*, vol. 36, no. 1, pp. 86–97, 2016.
- [3] Zisimopoulos O. et al., “Deep Learning Temporal Convolutional Networks for Surgical Gesture Recognition,” in *Med. Image Comput. Comput. Assist. Interv. (MICCAI)*, pp. 643–650, 2018.
- [4] Reiley, Christopher, Lin, Henry, David D., and Hager, Gregory D, “ Review of methods for objective surgical skill evaluation,” *Surgical Endoscopy*, 25(2), 2011.
- [5] Hutchinson R. et al., “Analysis of Executional and Procedural Errors in Dry Lab Robotic Surgery Experiments,” *Int. J. Comput. Assist. Radiol. Surg.*, vol. 15, pp. 1153–1161, 2020.
- [6] Yixin Gao, S. Swaroop Vedula et al., “The JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS): A Surgical Activity Dataset for Human Motion Modeling,” In *Modeling and Monitoring of Computer Assisted Interventions (M2CAI) – MICCAI Workshop*, 2014.
- [7] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk & Yoshua Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [9] Aston Zhang, Zachary C. Lipton, Mu Li & Alexander J. Smola, “Dive into Deep Learning,” *Chapter Gated Recurrent Units (GRU)*, 2024.
- [10] Jialang Xu et al., “SEDMamba: Enhancing selective state space modelling with bottleneck mechanism and fine-to-coarse temporal fusion for efficient error detection in robot-assisted surgery,” *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 232–239, Jan. 2025.
- [11] Kai Chen et al., “A Real-Time Approach for Surgical Activity Recognition and Prediction Based on Transformer Models in Robot-Assisted Surgery,” *International Journal of Computer Assisted Radiology and Surgery*, vol. 20, no. 1, pp. 1–10, 2025.

- [12] Lin H. et al., “Think Step by Step: Chain of Gesture Prompting for Error Detection in Robotic Surgical Videos,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 2701–2710, 2024.
- [13] Dionysis Anastasiou et al., “Keep Your Eye on the Best: Contrastive Regression Transformer for Skill Assessment in Robotic Surgery,” IEEE Robotics and Automation Letters, vol. 8, no. 2, pp. 1–8, 2023.
- [14] Yipeng Jin et al., “Recognition and Prediction of Surgical Gestures and Trajectories Using Transformer Models in Robot-Assisted Surgery,” arXiv preprint arXiv:2212.01683, 2022.
- [15] Tao L. et al., “Runtime Detection of Executional Errors in Robot Assisted Surgery,” IEEE Transactions on Medical Robotics and Bionics, vol. 3, no. 1, pp. 113–124, 2021.
- [16] Isabell Funke et al., “Using 3D Convolutional Neural Networks to Learn Spatiotemporal Features for Automatic Surgical Gesture Recognition in Video,” Medical Image Computing and Computer-Assisted Intervention – MICCAI, pp. 467–475, 2019.
- [17] Syed Zia and Y. Guo, “Novel Evaluation of Surgical Activity Recognition Models Using Task-Based Efficiency Metrics,” Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI), pp. 1–9, 2019.
- [18] Sarikaya D. et al., “Real Time Context Aware Detection of Unsafe Events in Robot Assisted Surgery,” International Journal of Computer Assisted Radiology and Surgery, vol. 14, no. 7, pp. 1181–1188, 2019.
- [19] S. M. Kamrul Hasan et al., “U-NetPlus: A Modified Encoder-Decoder U-Net Architecture for Semantic and Instance Segmentation of Surgical Instruments,” Proc. International Conference on Electrical, Computer and Communication Engineering (ECCE), pp. 1–6, 2019.
- [20] Yipeng Jin et al., “Real-Time Surgical Instrument Detection in Robot-Assisted Surgery Using a Convolutional Neural Network Cascade,” IET Healthcare Technology Letters, vol. 6, no. 1, pp. 1–6, 2019.
- [21] Jinliang Zang, Le Wang, Ziyi Liu, Qilin Zhang, Zhenxing Niu, Gang Hua & Nanning Zheng, “Attention-based Temporal Weighted Convolutional Neural Network for Action Recognition,” Proceedings of ICIP Workshops, 2018.
- [22] C. Lea, M. Flynn, R. Vidal, et al., “Temporal Convolutional Networks for Action Segmentation and Detection.” arXiv, Nov 2016.
- [23] Qingsong Wen, Liang Sun, Fan Yang et al., “Time Series Data Augmentation for Deep Learning: A Survey,” arXiv, Feb 2020.

- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser & Illia Polosukhin, “Attention Is All You Need,” NeurIPS, 2017.
- [25] Ogihara, A., Omata, M., Shidei, H. et al., “Intraoperative robotic surgical system-related problems in robot-assisted thoracoscopic surgery,” Gen Thorac Cardiovasc Surg 72, 593–598, 2024.
- [26] Karen Simonyan and Andrew Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” Advances in Neural Information Processing Systems (NeurIPS), 2014.
- [27] D. Psychogyios, M. Boels, R. Sparks, et al. “SAR-RARP50: Segmentation of Surgical Instrumentation and Action Recognition on Robot-Assisted Radical Prostatectomy,” arXiv preprint arXiv:2401.00496, 2024.

APPENDIX

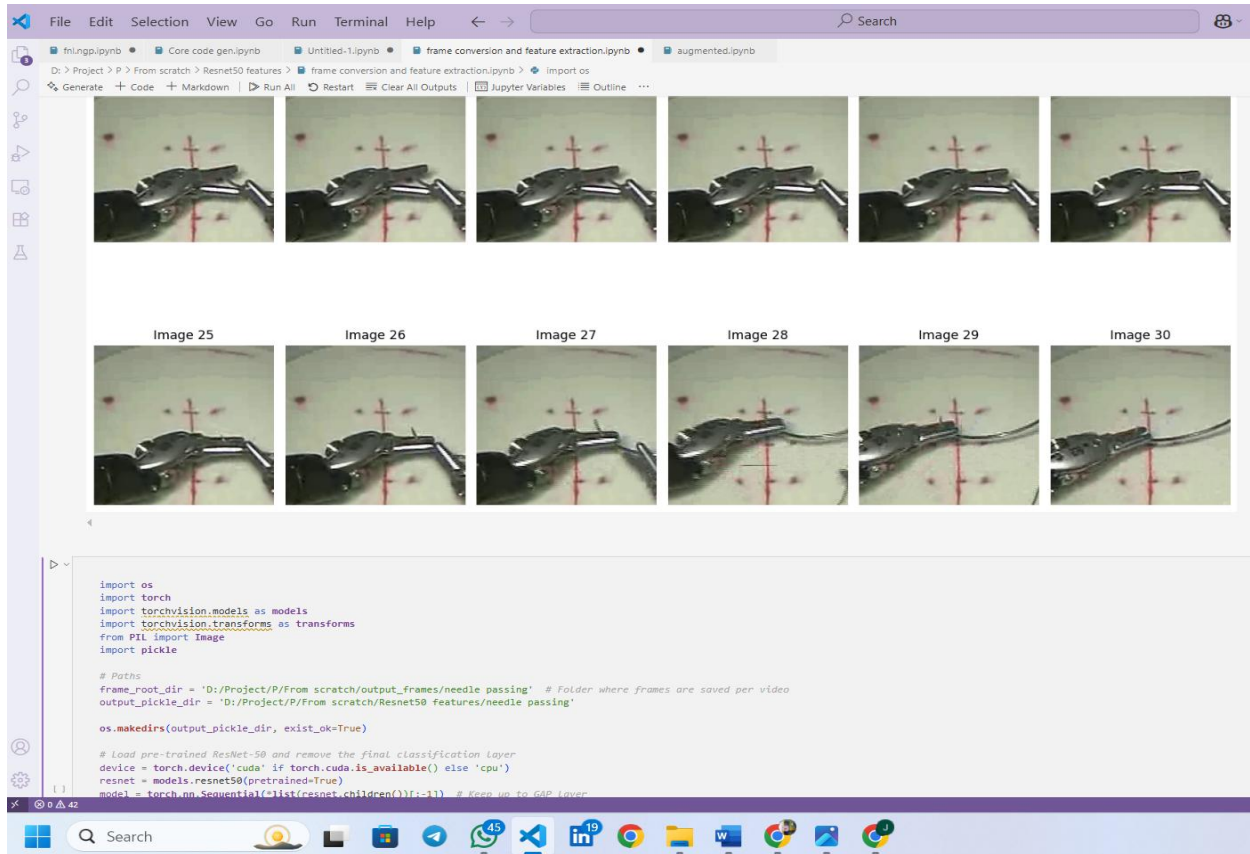


Figure i: Frame conversion and center resizing for standardization.

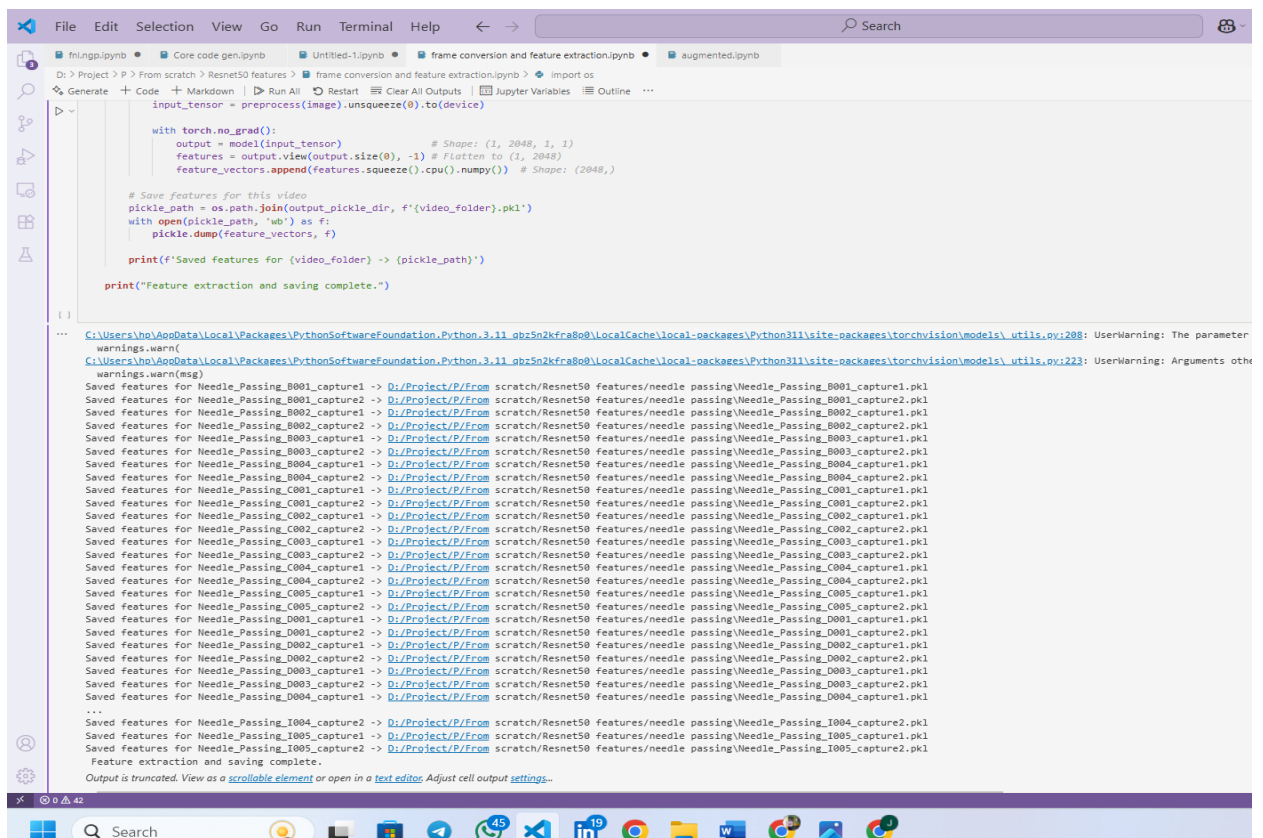


Figure ii: Frame-wise 2048-d feature vectors stored per video in pickle files.

```

File Edit Selection View Go Run Terminal Help
D:\Project> P> From scratch> fmlngp.ipynb> #Main Execution with Data Loading
Generate Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...

output_path = os.path.join(args.output_dir, f"{(base_name)_aug(idx+1)}.pkl")
with open(output_path, 'wb') as f:
    pickle.dump(augmented_data, f)
print(f"Saved augmented file: {output_path}")

print(f"Feature augmentation complete. Total files: {len(pickle_files) * args.num_augmentations}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Augment JIGSAWS dataset features for GESTURE-Net")
    parser.add_argument("--input_dir", type=str, default="D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/mx")
    parser.add_argument("--output_dir", type=str, default="D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd")
    parser.add_argument("--num_augmentations", type=int, default=2, help="Number of augmented versions per file")
    parser.add_argument("--noise_scale", type=float, default=0.05, help="Scale of Gaussian noise for features")
    parser.add_argument("--scale_range_min", type=float, default=0.9, help="Min scale for feature scaling")
    parser.add_argument("--scale_range_max", type=float, default=1.1, help="Max scale for feature scaling")
    parser.add_argument("--max_shift", type=int, default=2, help="Max temporal shift for features")
    # Use parse_known_args to handle Jupyter's extra arguments
    args, unknown = (parser.parse_known_args() if 'ipykernel_launcher' in sys.argv[0] else (parser.parse_args(), []))

    main(args)

[2]
...
Found 67 files. Augmenting features...
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B001_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B001_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B002_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B002_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B003_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B003_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B004_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_B004_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C001_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C001_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C002_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C002_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C003_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C003_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C004_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C004_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C005_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_C005_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D001_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D001_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D002_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D002_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D003_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Needle_Passing_D003_aug2.pkl
...
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Suturing_I004_aug2.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Suturing_I005_aug1.pkl
Saved augmented file: D:/Project/P/Githubdata/Chain-of-Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd/Suturing_I005_aug2.pkl
Feature augmentation complete. Total files: 134
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Figure iii: Augmentation techniques applied directly to frame-wise feature vectors.

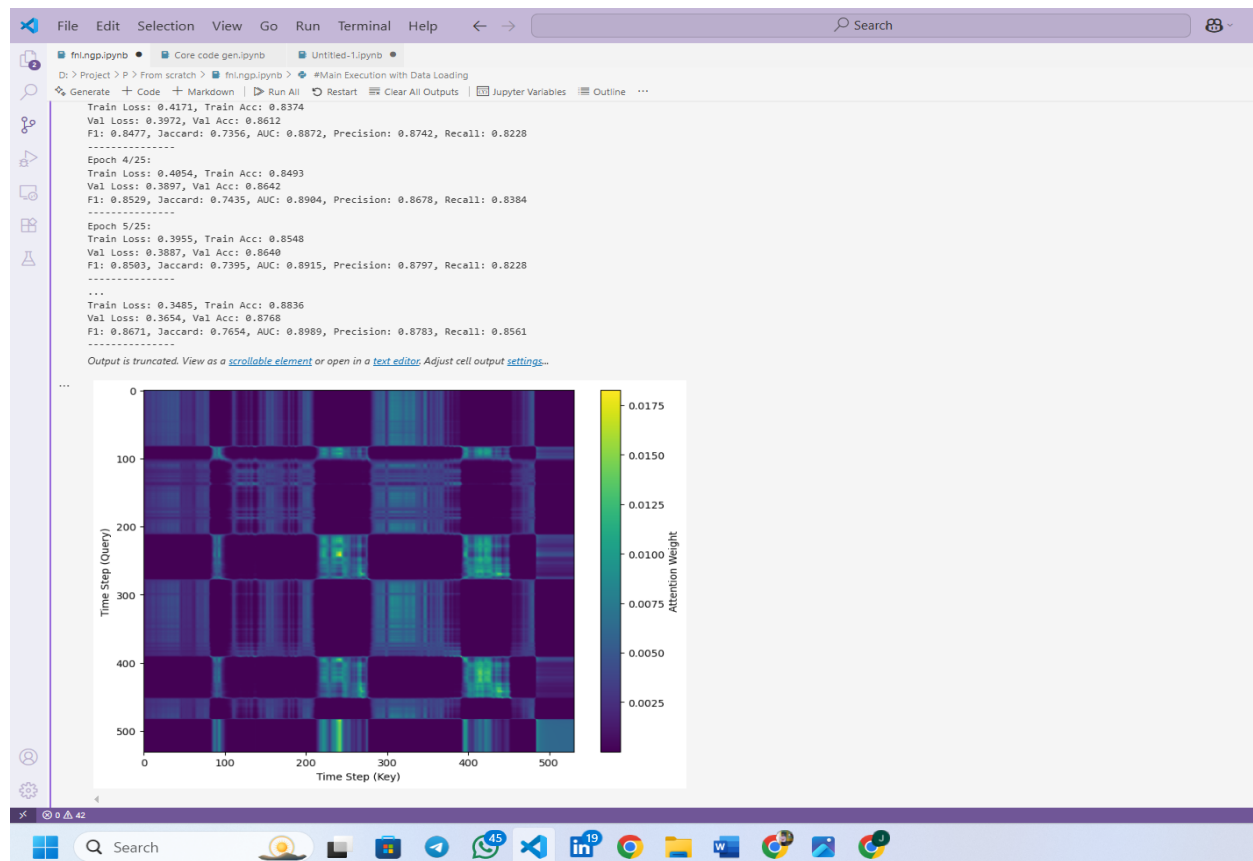


Figure iv: Self-attention map over temporal features.

Source Code

Frame conversion and center resizing

```
import cv2
import os

video_folder = 'D:/Project/P/Dataset/JIGSAWS data/Needle_Passing/video'
output_folder = 'D:/Project/P/From scratch/output_frames/needle passing'
fps_target = 5
crop_size = 224
os.makedirs(output_folder, exist_ok=True)
for video_name in os.listdir(video_folder):
    if not video_name.lower().endswith((''.mp4', '.avi', '.mov', '.mkv')):
        continue
    video_path = os.path.join(video_folder, video_name)
    cap = cv2.VideoCapture(video_path)
    original_fps = cap.get(cv2.CAP_PROP_FPS)
    if original_fps < fps_target:
        print(f'Skipping {video_name}: original FPS < {fps_target}')
        continue
    frame_interval = int(original_fps // fps_target)
    video_base = os.path.splitext(video_name)[0]
    video_output_dir = os.path.join(output_folder, video_base)
    os.makedirs(video_output_dir, exist_ok=True)
    count = 0
    saved = 0
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        if count % frame_interval == 0:
            h, w, _ = frame.shape
            top = max(0, h // 2 - crop_size // 2)
            left = max(0, w // 2 - crop_size // 2)
            cropped = frame[top:top + crop_size, left:left + crop_size]
            save_path = os.path.join(video_output_dir, f'frame_{saved:04d}.jpg')
            cv2.imwrite(save_path, cropped)
            saved += 1
        count += 1
    cap.release()
print("All frames extracted and saved in separate folders.")
```

```
# Augmentation techniques applied directly to frame-wise feature vectors
```

```
import numpy as np
```

```
import pickle
```

```
import os
```

```
import glob
```

```
import argparse
```

```
import sys
```

```
from scipy.ndimage import shift
```

```
def add_gaussian_noise(features, noise_scale=0.05):
```

```
    """Add Gaussian noise to feature vectors."""
```

```
    noise = np.random.normal(0, noise_scale, features.shape)
```

```
    return features + noise
```

```
def scale_features(features, scale_range=(0.9, 1.1)):
```

```
    """Randomly scale feature vectors."""
```

```
    scale = np.random.uniform(scale_range[0], scale_range[1])
```

```
    return features * scale
```

```
def temporal_jitter(features, max_shift=2):
```

```
    """Apply temporal jittering by shifting features."""
```

```
    shift_amount = np.random.randint(-max_shift, max_shift + 1)
```

```
    return shift(features, [shift_amount, 0], mode='nearest')
```

```
def augment_data(data, noise_scale=0.05, scale_range=(0.9, 1.1), max_shift=2):
```

```
    features = data['feature']
```

```
    features = add_gaussian_noise(features, noise_scale)
```

```
    features = scale_features(features, scale_range)
```

```
    features = temporal_jitter(features, max_shift)
```

```
    return {
```

```
        'feature': features,
```

```
        'gesture_GT': data['gesture_GT'],
```

```
        'error_GT': data['error_GT']
```

```
    }
```

```
def main(args):
```

```
    pickle_files = glob.glob(os.path.join(args.input_dir, "*.pkl"))
```

```
    if not pickle_files:
```

```
        raise FileNotFoundError(f"No pickle files found in {args.input_dir}")
```

```
    os.makedirs(args.output_dir, exist_ok=True)
```

```
    print(f"Found {len(pickle_files)} files. Augmenting features...")
```

```
    for file_path in pickle_files:
```

```
        with open(file_path, 'rb') as f:
```

```
            data = pickle.load(f)
```

```
        for aug_idx in range(args.num_augmentations):
```

```
            augmented_data = augment_data(
```

```
                data,
```

```
                noise_scale=args.noise_scale,
```

```
                scale_range=(args.scale_range_min, args.scale_range_max),
```

```
                max_shift=args.max_shift
```

```
            )
```

```
base_name = os.path.basename(file_path).replace('.pkl', '')
output_path = os.path.join(args.output_dir, f'{base_name}_{aug_idx+1}.pkl')
with open(output_path, 'wb') as f:
    pickle.dump(augmented_data, f)
print(f"Saved augmented file: {output_path}")

print(f"Feature augmentation complete. Total files: {len(pickle_files) *
args.num_augmentations}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Augment JIGSAWS dataset features for
GESTURE-Net")
    parser.add_argument('--input_dir', type=str, default='D:/Project/P/Githubdata/Chain-of-
Gesture-main/dataset/setting_f1/LOSO/All/temp/mx')
    parser.add_argument('--output_dir', type=str, default='D:/Project/P/Githubdata/Chain-of-
Gesture-main/dataset/setting_f1/LOSO/All/temp/augmentd')
    parser.add_argument('--num_augmentations', type=int, default=2, help="Number of
augmented versions per file")
    parser.add_argument('--noise_scale', type=float, default=0.05, help="Scale of Gaussian noise
for features")
    parser.add_argument('--scale_range_min', type=float, default=0.9, help="Min scale for feature
scaling")
    parser.add_argument('--scale_range_max', type=float, default=1.1, help="Max scale for
feature scaling")
    parser.add_argument('--max_shift', type=int, default=2, help="Max temporal shift for
features")
    args, unknown = (parser.parse_known_args() if 'ipykernel_launcher' in sys.argv[0] else
(parser.parse_args(), []))
    main(args)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from sklearn.metrics import f1_score, accuracy_score, jaccard_score, confusion_matrix,
roc_auc_score, precision_score, recall_score
from sklearn.model_selection import GroupKFold
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import argparse
import os
import glob
import sys

GESTURES = [
    "G1: Reaching for needle with right hand",
    "G2: Positioning needle",
    "G3: Pushing needle through tissue",
```

"G4: Transferring needle from left to right",
"G5: Pulling suture with left hand",
"G6: Pulling suture with right hand",
"G7: Orienting needle",
"G8: Reaching for needle with left hand",
"G9: Making C loop",
"G10: Pushing needle with left hand",
"G11: Scooping suture",
"G12: Tensioning suture",
"G13: Cutting suture",
"G14: Dropping suture",
"G15: Idle"

]

#Model Definitions

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self, d_k):
        super().__init__()
        self.d_k = d_k

    def forward(self, Q, K, V):
        scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(self.d_k,
dtype=torch.float32))
        attn = torch.softmax(scores, dim=-1)
        output = torch.matmul(attn, V)
        return output, attn

class GESTURE_NET(nn.Module):
    def __init__(self, input_dim=2048, num_gestures=15, num_classes=2, window_size=5,
gru_hidden_size=64, attn_dim=64, gesture_embed_dim=32):
        super().__init__()
        self.conv1d = nn.Conv1d(input_dim, 128, kernel_size=window_size,
padding=window_size//2)
        self.relu = nn.ReLU()
        self.gru = nn.GRU(128, gru_hidden_size, batch_first=True)
        self.attention = ScaledDotProductAttention(d_k=attn_dim)
        self.gesture_embedding = nn.Embedding(num_gestures, gesture_embed_dim)
        self.agnostic_fc = nn.Conv1d(gru_hidden_size, num_classes, kernel_size=1)
        self.fusion = nn.Linear(gru_hidden_size + gesture_embed_dim, gru_hidden_size)
        self.aware_fc = nn.Conv1d(gru_hidden_size, num_classes, kernel_size=1)
```

```
self.weighting = nn.Linear(gru_hidden_size, 2)

def forward(self, features, gestures, return_attn=False):
    features = features.transpose(1, 2)
    temporal_features = self.relu(self.conv1d(features)).transpose(1, 2)
    gru_out, _ = self.gru(temporal_features)
    attn_out, attn_weights = self.attention(gru_out, gru_out, gru_out)

    agnostic_features = attn_out.transpose(1, 2)
    agnostic_output = self.agnostic_fc(agnostic_features).transpose(1, 2)

    gesture_embed = self.gesture_embedding(gestures.long())
    combined = torch.cat([attn_out, gesture_embed], dim=-1)
    fused_features = self.relu(self.fusion(combined))
    aware_features = fused_features.transpose(1, 2)
    aware_output = self.aware_fc(aware_features).transpose(1, 2)

    weights = self.weighting(attn_out)
    weights = torch.softmax(weights, dim=-1)
    w_agnostic, w_aware = weights.split(1, dim=-1)
    final_output = w_agnostic * agnostic_output + w_aware * aware_output

    if return_attn:
        return final_output, attn_weights, w_agnostic, w_aware
    return final_output
```

#Dataset Definition

```
class JIGSAWSFeatureDataset(Dataset):
    def __init__(self, pickle_files):
        self.data = []
        for file in pickle_files:
            with open(file, 'rb') as f:
                data = pickle.load(f)
                features = torch.tensor(data['feature'], dtype=torch.float32)
                labels = torch.tensor(data['error_GT'], dtype=torch.long)
                gestures = torch.tensor(data['gesture_GT'], dtype=torch.long)
                self.data.append((features, labels, gestures))
```

```
def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    return self.data[idx][0], self.data[idx][1].float(), self.data[idx][2]
```

#Plotting Utilities

```
def plot_attn_map(attn_weights):
    plt.figure(figsize=(8, 6))
    plt.imshow(attn_weights.squeeze().detach().cpu().numpy(), cmap='viridis', aspect='auto')
    plt.xlabel('Time Step (Key)')
    plt.ylabel('Time Step (Query)')
    plt.colorbar(label='Attention Weight')
    plt.tight_layout()
    plt.show()

def plot_gestures(gestures, pred_gestures, gesture_labels):
    # Convert tensors to numpy and take the first sample
    gestures = gestures.squeeze().detach().cpu().numpy()
    pred_gestures = pred_gestures.squeeze().detach().cpu().numpy()
    time_steps = np.arange(len(gestures))

    # Define colors for each gesture using tab20 colormap
    cmap = plt.get_cmap('tab20')
    colors = [cmap(i / len(gesture_labels)) for i in range(len(gesture_labels))]

    # Create figure with two subplots
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 4), sharex=True)

    # Plot ground truth gestures
    for i in range(len(gestures)):
        gesture_idx = int(gestures[i])
        if 0 <= gesture_idx < len(gesture_labels):
            ax1.plot([i, i+1], [1, 1], color=colors[gesture_idx], linewidth=10, solid_capstyle='butt')

    # Plot predicted gestures
    for i in range(len(pred_gestures)):
        gesture_idx = int(pred_gestures[i])
        if 0 <= gesture_idx < len(gesture_labels):
            ax2.plot([i, i+1], [1, 1], color=colors[gesture_idx], linewidth=10, solid_capstyle='butt')

    # Customize axes
    ax1.set_yticks([])
    ax1.set_title('Ground Truth Gestures')
    ax2.set_yticks([])
    ax2.set_title('Predicted Gestures')
    ax2.set_xlabel('Time Step')
```

```
# Create legend
from matplotlib.lines import Line2D
legend_elements = [Line2D([0], [0], color=colors[i], lw=4, label=gesture_labels[i]) for i in
range(len(gesture_labels))]
fig.legend(handles=legend_elements, loc='center left', bbox_to_anchor=(1, 0.5), ncol=1)

plt.tight_layout()
plt.show()
def plot_dynamic_weights(w_agnostic, w_aware, gestures):
    w_agnostic = w_agnostic.squeeze().detach().cpu().numpy()
    w_aware = w_aware.squeeze().detach().cpu().numpy()
    gestures = gestures.squeeze().detach().cpu().numpy()
    time_steps = np.arange(len(w_agnostic))

    plt.figure(figsize=(12, 4))
    plt.stackplot(time_steps, w_agnostic, w_aware, labels=['Agnostic', 'Aware'],
colors=['#FF9999', '#66B2FF'], alpha=0.8)
    plt.xlabel('Time Step')
    plt.ylabel('Weight')
    plt.legend(loc='upper right')
    for t in range(0, len(time_steps), len(time_steps)//10):
        if 0 <= gestures[t] < len(GESTURES):
            plt.text(t, 0.1, f"G{gestures[t]+1}", rotation=45, fontsize=8)
    plt.tight_layout()
    plt.show()

def plot_loss_accuracy_curves(train_losses, val_losses, train_accs, val_accs):
    epochs = range(1, len(train_losses) + 1)
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_losses, label='Train Loss')
    plt.plot(epochs, val_losses, label='Val Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_accs, label='Train Acc')
    plt.plot(epochs, val_accs, label='Val Acc')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_confusion_matrix(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(4, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0, 1], yticklabels=[0, 1])
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.tight_layout()
    plt.show()
```

#Model Training

```
def train_model(args, train_loader, val_loader):
    device = torch.device(f'cuda:{args.gpu}' if torch.cuda.is_available() else "cpu")
    model = GESTURE_NET(input_dim=2048, num_gestures=15).to(device)
    optimizer = optim.Adam(model.parameters(), lr=args.lr)
    criterion = nn.BCEWithLogitsLoss()

    best_f1 = 0
    train_losses, val_losses = [], []
    train_accs, val_accs = [], []

    for epoch in range(args.epochs):
        model.train()
        train_loss = 0
        train_preds, train_labels = [], []
        for features, labels, gestures in train_loader:
            features, labels, gestures = features.to(device), labels.to(device), gestures.to(device)
            optimizer.zero_grad()
            outputs = model(features, gestures)
            labels_onehot = torch.stack([1 - labels, labels], dim=-1)
            loss = criterion(outputs, labels_onehot)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()

            preds = torch.sigmoid(outputs).argmax(dim=-1).cpu().numpy()
            train_preds.extend(preds.flatten())
            train_labels.extend(labels.cpu().numpy().flatten())

        train_loss /= len(train_loader)
        train_acc = accuracy_score(train_labels, train_preds)
        train_losses.append(train_loss)
        train_accs.append(train_acc)

        model.eval()
        val_loss = 0
        val_preds, val_labels, val_probs = [], [], []
        with torch.no_grad():
            for i, (features, labels, gestures) in enumerate(val_loader):
                features, labels, gestures = features.to(device), labels.to(device), gestures.to(device)
                outputs, attn_weights, w_agnostic, w_aware = model(features, gestures,
                return_attn=True)
                labels_onehot = torch.stack([1 - labels, labels], dim=-1)
                val_loss += criterion(outputs, labels_onehot).item()

                probs = torch.sigmoid(outputs)[: , :, 1].cpu().numpy()
                preds = probs > 0.5
                val_preds.extend(preds.flatten())
```



```
val_labels.extend(labels.cpu().numpy().flatten())
val_probs.extend(probs.flatten())

if epoch == args.epochs - 1 and i == 0:
    plot_attention_map(attn_weights[0])
    plot_dynamic_weights(w_agnostic[0], w_aware[0], gestures[0])
    # Placeholder for predicted gestures; replace with actual predictions if available
    pred_gestures = gestures # TODO: Replace with model gesture predictions
    plot_gestures(gestures[0], pred_gestures[0], GESTURES)

val_loss /= len(val_loader)
f1 = f1_score(val_labels, val_preds)
val_acc = accuracy_score(val_labels, val_preds)
jaccard = jaccard_score(val_labels, val_preds)
auc = roc_auc_score(val_labels, val_probs)
precision = precision_score(val_labels, val_preds, zero_division=0)
recall = recall_score(val_labels, val_preds, zero_division=0)

val_losses.append(val_loss)
val_accs.append(val_acc)

print(f'Epoch {epoch+1}/{args.epochs}:')
print(f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}')
print(f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}')
print(f'F1: {f1:.4f}, Jaccard: {jaccard:.4f}, AUC: {auc:.4f}, Precision: {precision:.4f},
Recall: {recall:.4f}')
print("-" * 15)

if f1 > best_f1:
    best_f1 = f1
    torch.save(model.state_dict(), os.path.join(args.output_dir, "gesture_net_best.pth"))

plot_confusion_matrix(val_labels, val_preds)
plot_loss_accuracy_curves(train_losses, val_losses, train_accs, val_accs)
return best_f1, val_acc, jaccard, auc, precision, recall

#Main Execution with Data Loading

def main(args):
    pickle_files = glob.glob(os.path.join(args.data_dir, "*.pkl"))
    if not pickle_files:
        raise FileNotFoundError(f"No pickle files found in {args.data_dir}")

    os.makedirs(args.output_dir, exist_ok=True)
    subject_ids = [os.path.basename(f).split()[-1].replace('.pkl', '') for f in pickle_files]

    gkf = GroupKFold(n_splits=5)
    results = []
```

```

for train_idx, val_idx in gkf.split(pickle_files, groups=subject_ids):
    train_files = [pickle_files[i] for i in train_idx]
    val_files = [pickle_files[i] for i in val_idx]

    train_dataset = JIGSAWSFeatureDataset(train_files)
    val_dataset = JIGSAWSFeatureDataset(val_files)

    train_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=1, shuffle=False)

    metrics = train_model(args, train_loader, val_loader)
    results.append(metrics)

f1_scores, acc_scores, jaccard_scores, auc_scores, precision_scores, recall_scores =
zip(*results)
print("\nFinal Evaluation Across Folds:")
print(f'F1: {np.mean(f1_scores):.4f} ± {np.std(f1_scores):.4f}')
print(f'Accuracy: {np.mean(acc_scores):.4f} ± {np.std(acc_scores):.4f}')
print(f'Jaccard: {np.mean(jaccard_scores):.4f} ± {np.std(jaccard_scores):.4f}')
print(f'AUC: {np.mean(auc_scores):.4f} ± {np.std(auc_scores):.4f}')
print(f'Precision: {np.mean(precision_scores):.4f} ± {np.std(precision_scores):.4f}')
print(f'Recall: {np.mean(recall_scores):.4f} ± {np.std(recall_scores):.4f}')

# Define arguments
parser = argparse.ArgumentParser(description="GESTURE-Net for Surgical Error Detection")
parser.add_argument('--gpu', type=int, default=0)
parser.add_argument('--epochs', type=int, default=5)
parser.add_argument('--lr', type=float, default=7e-5)
parser.add_argument('--data_dir', type=str, default='D:/Project/P/Githubdata/Chain-of-Gesture-
main/dataset/setting_f1/LOSO/All/temp/augmented')
parser.add_argument('--output_dir', type=str, default='./modelnpop')
args, unknown = parser.parse_known_args() if 'ipykernel_launcher' in sys.argv[0] else
parser.parse_args()

# Run main
main(args)

```